

Meta-PBS: Compact High-Precision Programmable Bootstrapping

Shihe Ma¹[0009–0006–9212–3260], Tairong Huang², Anyu Wang^{2,3,4,5,6}[0000–0002–1086–0288] (✉), Changtong Xu⁷, Tao Wei⁷, and Xiaoyun Wang^{2,3,4,5,6,8}

¹ Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China, msh24@mails.tsinghua.edu.cn

² Institute for Advanced Study, Tsinghua University, Beijing, China, huangtr@mail.tsinghua.edu.cn, {anyuwang,xiaoyunwang}@tsinghua.edu.cn

³ State Key Laboratory of Cryptography and Digital Economy Security, Tsinghua University, Beijing, China

⁴ School of Cryptologic Science and Engineering, Shandong University, Jinan, China

⁵ Zhongguancun Laboratory, Beijing, China

⁶ National Financial Cryptography Research Center, Beijing, China

⁷ Ant Group, {xuchangtong.xct,lenx.wei}@antgroup.com

⁸ Shandong Institute of Blockchain, Shandong, China

Abstract. Currently, most FHE schemes realize bootstrapping through the linear-decrypt-then-round paradigm. For the programmable bootstrapping (PBS) of TFHE, this means the lookup table (LUT) needs a redundancy of $O(\sqrt{N})$ to be able to remove the modulus switching noise, which limits the plaintext modulus of PBS to $O(\sqrt{N})$. We remove this requirement for redundancy by proposing the Meta-PBS framework, which allows us to start with under-redundant or non-redundant LUTs. Meta-PBS iteratively blind-rotates the LUT, during which the LUT redundancy gradually increases. The bootstrapping outputs the correct result when the redundancy eventually exceeds the noise bound. Asymptotically, Meta-PBS requires $O(1)$ blind rotations in dimension N to evaluate a negacyclic function modulo $2N$, whereas PBS needs $O(\sqrt{N})$ blind rotations. Meta-PBS also enjoys an additive noise growth, allowing for more homomorphic arithmetic on bootstrapped ciphertext. We modified Meta-PBS to support the simultaneous evaluation of multiple LUTs on the same ciphertext and/or arbitrary LUTs. According to our implementation, when evaluating a 12-bit negacyclic function, Meta-PBS outperforms EBS (PKC’23) by 79 times. When evaluating an arbitrary function on an 8-bit LWE ciphertext, Meta-PBS reduces the running time of the Refined LHE (CCS’25) by half while allowing for a 27 times larger post-bootstrap linear combination.

1 Introduction

Fully Homomorphic Encryption (FHE) is a powerful cryptographic tool for performing arbitrary computation over encrypted data [27]. Since all FHE construction base their security on noisy encryptions, Gentry’s bootstrapping method,

which reduces the noise level homomorphically, is essential for preventing noise blowup and realizing infinite homomorphic computation [13]. Bootstrapping removes the original ciphertext noise by homomorphically evaluating the decryption circuit. For exact FHE schemes based on Learning With Errors (LWE) [26] and Ring-LWE (RLWE) [23], their decryption follows the linear-decrypt-and-round paradigm, where the plaintext needs to be encoded *redundantly* onto a larger space for successful noise removal during decryption. As a typical example, when $m \in \mathbb{Z}_t$ is encoded as $\lfloor \frac{q}{t} m \rfloor \in \mathbb{Z}_q$, the decryption (or bootstrapping) succeeds as long as the noise size is smaller than $\frac{q}{2t}$.

Such redundant encoding means bootstrapping needs to operate over a larger space than the plaintext space. For BFV [4,12] and BGV [5], this translates to a larger plaintext modulus during bootstrapping [15,18]. For FHEW [11]/TFHE [9], where the rounding is performed over the exponent of X in $\mathbb{Z}[X]/(X^N + 1)$, the larger space where the encoded message resides cannot exceed $\mathbb{Z}_{2N}$. The noise during decryption is no smaller than a modulus switching noise [10], which has a bound of $O(\sqrt{N})$ for a dense secret in N -dimensional LWE/RLWE. Thus, the plaintext modulus t in TFHE bootstrapping is bounded by $O(\sqrt{N})$.

TFHE bootstrapping is a powerful tool for non-polynomial function evaluation because it is able to evaluate a lookup table (LUT) simultaneously (i.e., programmable). However, the limited bootstrapping precision of TFHE has become a major difficulty in its adoption in real-world applications. To address this limitation, existing approaches either represent a plaintext using multiple ciphertexts or increase the RLWE dimension to realize high-precision computation beyond the native plaintext space. The leveled homomorphic evaluation (LHE) mode of TFHE encrypts plaintext data bit-wise, using external products for binary arithmetic and circuit bootstrapping (CBS) for noise control [9,30,29]. Tree-based/chaining method encrypts the decomposed digits of a high-precision message into multiple ciphertexts, using multiple bootstrapping and LWE-to-RLWE packings to perform large-precision LUT evaluation [16,28]. Extended Bootstrapping (EBS) uses an increased RLWE dimension N to allow for a larger t while using a subring secret to prevent noise blowup and achieve better parallelism [21].

1.1 Our Contributions

We propose Meta-PBS, a generalization of programmable bootstrapping that can bootstrap under-redundant (or even non-redundant) ciphertexts through iterative calls to the basic TFHE bootstrapping. This enables Meta-PBS to evaluate (negacyclic) LUTs with up to $2N$ entries without relying on decomposed ciphertexts or increased RLWE dimensions. From the asymptotical perspective, Meta-PBS requires $O(1)$ calls to bootstrapping in dimension N to evaluate a size $2N$ negacyclic LUT, while PBS and EBS need $O(\sqrt{N})$ bootstrappings to evaluate an LUT this size. We also design Meta-PBSManyLUTs to support multi-output PBS and Meta-WoPPBSManyLUTs for multi-output nonnegacyclic PBS. Meta-PBS and its variants enjoy an additive noise growth, i.e., the output noise is the sum of several blind rotation noises and RLWE packing noises. In contrast,

the blind rotation noise variance in CBS in Refined LHE and the tree-based method (with basis B_{tree}) are multiplied by $O(N)$ and $O(t^2 B_{\text{tree}})$, respectively. Thus, Meta-PBS has more after-bootstrap capacity for further homomorphic operations, for example, constant multiplications or ciphertext additions.

Besides its theoretical importance, Meta-PBS is also practically competent. Our experiments demonstrate that Meta-PBS can evaluate a 12-bit negacyclic LUT in 156 ms, which is 82.4 times faster than EBS [21]. Moreover, when given an 8-bit LWE ciphertext, Meta-WoPPBSManyLUTs is capable of evaluating an arbitrary LUT with capacity enough for $\mathbb{Z}$ -linear combination of 25000 ciphertexts, while Refined LHE [29] doubles the running time and supports less than 1000 additions.

1.2 Technical Overview

Before introducing our techniques, we briefly review the original TFHE programmable bootstrapping. The input LWE ciphertext has the form of $c = (\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + \frac{q}{t}m + e) \in \mathbb{Z}_q^{n+1}$, where $\mathbf{s}, m, e$ are the secret, plaintext, and LWE noise respectively. The RLWE is instantiated over $R_q = \mathbb{Z}_q/(X^N + 1)$. Before bootstrapping, c is first modulus-switched into an LWE ciphertext $c' = (\mathbf{a}', b') \in \mathbb{Z}_{2N}^{n+1}$. A linear decryption of c' now becomes $\frac{2N}{t}m + e' = \frac{2N}{t}m + \lfloor \frac{2N}{q}e \rfloor + e_{\text{ms}}$, with e_{ms} as the modulus switching error. To evaluate a function $f : \mathbb{Z}_t \rightarrow \mathbb{Z}_t$ satisfying negacyclicity $f(x + t/2) = -f(x)$, TFHE bootstrapping homomorphically multiplies the test vector $\text{TV} = X^{-N/t} \sum_{i=0}^{t/2-1} \left(\lfloor \frac{q}{t} f(i) \rfloor X^{2Ni/t} \cdot \sum_{j=0}^{2N/t-1} X^j \right) \in R_q$ by $X^{-b + \langle \mathbf{a}, \mathbf{s} \rangle} = X^{-\frac{2N}{t}m - e'}$. If $|e'| < \frac{N}{t}$, the resulting RLWE ciphertexts encrypts $\lfloor \frac{q}{t} f(m) \rfloor$ in its constant term. This process is called blind rotation.

We reinterpret TV and the blind-rotated TV as polynomials whose terms have degrees in $[-N/2 - 1, N/2]$. This is equivalent to reinterpreting $a_i X^i$ as $-a_i X^{i-N}$ for $N/2 \leq i \leq N-1$ while keeping the other terms untouched. Under this interpretation, the $2N/t$ coefficients of TV around the constant term are equal to $\lfloor \frac{q}{t} f(m) \rfloor$, as depicted in Figure 1. Notably, the redundancy in TV prevents the perturbation from corrupting the constant term.

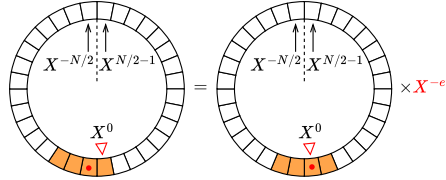


Fig. 1: PBS with $N = 32, t = 16$ and a perturbation of $e' = 1$. The circles on the LHS/RHS depict the blind rotation result w/ and w/o. the perturbation. Coefficients equal to $\lfloor \frac{q}{t} f(m) \rfloor$ are painted in orange. The red dot marks the center of the orange area.

The key observation in Meta-PBS is that the noise in decryption, e' , is dominated by e_{ms} , whose information can be obtained from the remainders in modulus switching. Using this information, we could increase the redundancy-to-noise ratio in the blind-rotated LUT through iteratively eliminating the modulus switching noise, and the bootstrapping produces the correct result when this ratio surpasses one. This enables us to start with a non-redundant LWE ciphertext and LUT, i.e., $t = 2N$. Here, each iteration takes one LWE-to-RLWE packing and one blind rotation. The original TFHE bootstrapping is a special case of Meta-PBS where the initial redundancy-to-noise ratio is already above one, i.e., with zero iterations. This iterative process of refining the bootstrapping result is similar to Meta-BTS [2], where the bootstrapping precision is increased by iteratively eliminating the bootstrapping-added error.

We give a toy example of Meta-PBS with $t = 2N$ and one iteration (e.g., two blind rotations). For an LWE ciphertext $c = (\mathbf{a}, b) \in \mathbb{Z}_q^{n+1}$, let $\psi^*(c) = b - \langle \mathbf{a}, \mathbf{s} \rangle \in \mathbb{Z}$ be the linear decryption result of c without modular reduction. The inputs of Meta-PBS include an LWE ciphertext $c \in \mathbb{Z}_q^{n+1}$ that encrypts $m \in \mathbb{Z}_{2N}$ with noise e . To simplify the example, we additionally assume that $|e| < \frac{q}{4N\beta}$. The test vector for a negacyclic function f modulo $2N$ is constructed as $\text{TV} = \sum_{i=0}^{N-1} \frac{q}{2N} f(i) \cdot X^i$.

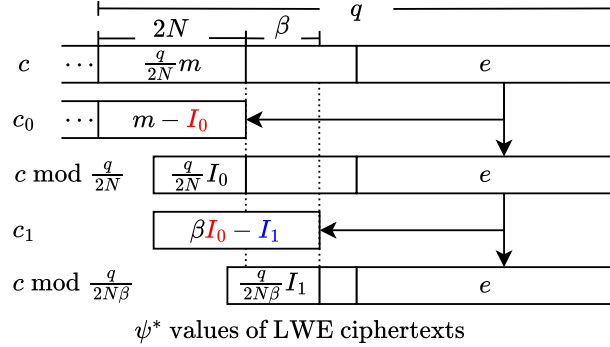


Fig. 2: Derivation of c_0 and c_1 from c . Each ciphertext is labeled with its ψ^* value.

For a power-of-two parameter $\beta \in \mathbb{Z}_{\geq 2}$, two LWE ciphertexts c_0 and c_1 are derived from c as

$$c_0 = \frac{c - (c \bmod q/(2N))}{q/(2N)} \in \mathbb{Z}^{n+1},$$

$$c_1 = \frac{(c \bmod q/(2N)) - (c \bmod q/(2N\beta))}{q/(2N\beta)} \in \mathbb{Z}^{n+1}.$$

c_0 is exactly the result of modulus-switching c to $2N$ that encrypts $m - I_0$. It will be proven later that $\psi^*(c_1) = \lfloor \frac{2N\beta}{q} e \rfloor + \beta \cdot I_0 - I_1$. I_0 (or I_1) is the overflow

of $c \bmod q/(2N)$ w.r.t. $q/(2N)$ (or $c \bmod q/(2N\beta)$ w.r.t. $q/(2N\beta)$). Both I_0 and I_1 follow centered Gaussian distributions with the same variance as e_{ms} and are upper-bounded by δ_I . The relationship between the ψ^* values of c , c_0 and c_1 is shown in Figure 2. We set $\beta = 4$, $\delta_I = 1$ and $I_0 = I_1 = 1$ in this example.¹

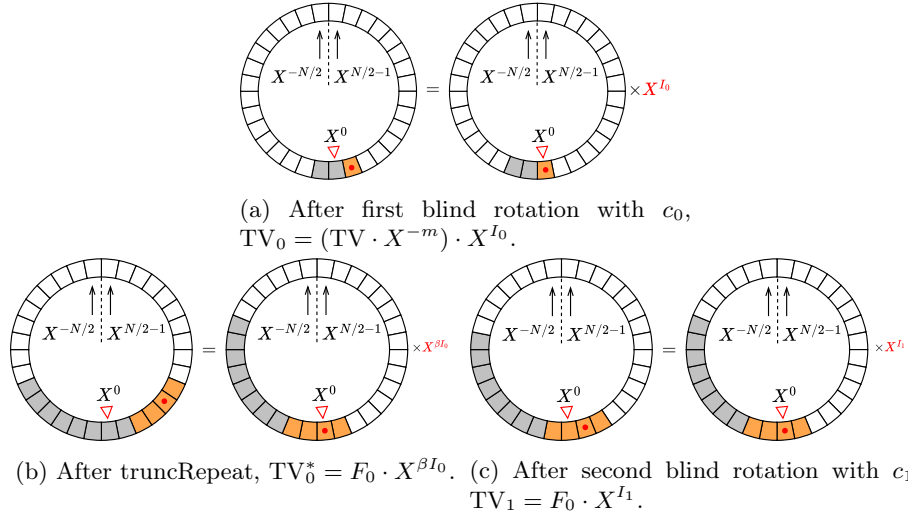


Fig. 3: Meta-PBS with one iteration for $t = 2N$, $|e| < \frac{q}{4N\beta}$, $\delta_I = 1$, $\beta = 4$ and $I_0 = I_1 = 1$. truncRepeat keeps the grey and orange areas in (a), and repeats each coefficient β times to produce the grey and orange areas in (b).

As depicted in Figure 3, Meta-PBS first blind-rotates TV with c_0 to obtain an RLWE encryption of $\text{TV}_0 = \text{TV} \cdot X^{-m+I_0}$. Then, it homomorphically applies a linear map called truncRepeat to TV_0 to obtain TV_0^* . $F_0 = \text{TV}_0^* \cdot X^{-\beta I_0}$ stores $\lfloor \frac{q}{t} f(m) \rfloor$ around the constant coefficient with a redundancy of β .

We provide an intuitive explanation of truncRepeat . Notice that the correct LUT entry in TV_0 (marked in orange) is at most δ_I positions away from the constant coefficient. Thus, truncRepeat first discards all coefficients in TV_0 that are more than δ_I positions away from the constant term, and then repeats the remaining terms by β times. Then, truncRepeat rotates the test vector by a constant offset so that the β redundant coefficients for $\lfloor \frac{q}{t} f(m) \rfloor$ are centered around the constant term of F_0 . $(2\delta_I + 1)\beta \leq N$ is required to ensure the correctness of truncRepeat , i.e., the duplicated coefficients do not overlap. Since truncRepeat is linear in the coefficients of TV_0 , one can always use the public functional key-switching from [9] to evaluate it².

¹ Later sections will show that Meta-PBS can work with a non-power-of-two β and a larger e . A typical setting of e is 1~2 bits smaller than $\frac{q}{2t}$.

² A more efficient approach to realize truncRepeat will be presented in Section 4.3.

As the final step of Meta-PBS, we blind-rotate TV_0^* with c_1 to obtain an encryption of $TV_1 = F_0 \cdot X^{\beta I_0 - \psi^*(c_1)} = F_0 \cdot X^{I_1}$. As long as $2\delta_I + 1 \leq \beta$, the constant term of $F_0 \cdot X^{I_1}$ equals $\lfloor \frac{q}{t} f(m) \rfloor$, and a sample extraction finishes the bootstrapping.

1.3 Concurrent Works

A parallel work by Nishimura et al. at CCS'25 also achieves high-precision PBS through iterative blind rotations [25]. However, their method, named **LargeLUT**, targets the situation where the input plaintext has its digits encrypted in multiple LWE ciphertexts, while Meta-PBS is performed on a single LWE ciphertext. Our method has the following advantages compared with **LargeLUT**.

Faster preprocessing. Due to the difference in input formats, **LargeLUT** needs to call PBS $O(\log^2 t)$ times to preprocess the input ciphertexts, while Meta-PBS only needs several calls to modulus switching for preprocessing, thus having a much lower overhead.

More efficient and linear transformations. Meta-PBS uses the more optimized TruncRepeat* for linear transformations, outperforming the regular PackingKS used in their work (see the comparison at the end of Section 4.3).

More flexible parameter choice. **LargeLUT** requires β to be a power of two, while this restriction is not present in Meta-PBS. Thus, **LargeLUT** suffers from a limited parameter selection space, and consequently needs more blind rotations than Meta-PBS under certain parameters.

Lower asymptotic complexity. The asymptotic complexity (in the number of blind rotations) of **LargeLUT** is estimated to be $O(\log^2 t)$, while Meta-PBS has a complexity of $O(1)$.

1.4 Paper Organization

Section 2 includes background knowledge used in this paper. Section 3 introduces core components for Meta-PBS. Section 4 describes the workflow of Meta-PBS and provides an analysis of correctness and efficiency. Section 5 modifies Meta-PBS to support LUT evaluation with multiple outputs and/or non-negacyclic LUTs. Section 6 presents the performance results of Meta-PBS and provides a comparison with other LUT evaluation methods.

2 Preliminaries

2.1 Basic Notations

For an integer $q \geq 2$, define $[q] = \mathbb{Z} \cap [0, q-1]$ and $[q]_{\text{sym}}$ as $\mathbb{Z} \cap [-\lfloor \frac{q}{2} \rfloor, \lceil \frac{q}{2} \rceil - 1]$, with $[2]_{\text{sym}} = \{0, 1\}$ as the only exception. We identify the representatives of

$\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ with $[q]_{\text{sym}}$. $[a]_q^{\text{sym}}$ and $[a]_q$ returns the unique element in $[q]_{\text{sym}}$ and $[q]$ that are congruent to a modulo q . $[\cdot]_q^{\text{sym}}, [\cdot]_q$ are applied to integer vectors and integer sets in the natural way. We use $\llbracket a, b \rrbracket$ to denote $\mathbb{Z} \cap [a, b]$. The diameter of an integer set S is defined as $\max S - \min S$.

For a (Laurent) polynomial $f(X)$, the coefficient corresponding to X^i is denoted by $f(X)[i]$. The support of a (Laurent) polynomial is defined as $\text{supp}(f(X)) = \{i \mid f(X)[i] \neq 0\}$. With N a power of two, the $2N$ -th cyclotomic ring is $R = \mathbb{Z}[X]/(X^N + 1)$. For an integer $q \geq 2$, $R_q = R/qR$. Elements in R are represented by polynomials with a support in $[N]$. For $A \in R$, its coefficient vector is denoted as $\mathbf{A} = (A[0], A[1], \dots, A[N-1])$. For $k \in \mathbb{Z}$, π_k is the homomorphism in $\mathbb{Z}[X, X^{-1}]$ sending X to X^k .

Gadget decomposition $G^{(B,l,q)}$ is parametrized by a base B , a level l , and the input modulus q . We assume $B^l \mid q$. For $a \in \mathbb{Z}_q$, let $\mathbf{g}^{(B,l,q)} = (\frac{q}{B^l}, \dots, \frac{q}{B^2}, \frac{q}{B}) \in \mathbb{Z}^l$, $G^{(B,l,q)}(a) \in \mathbb{Z}^l$ satisfies $G^{(B,l,q)}(a) \in [B]_{\text{sym}}^l$ and $a - \langle G^{(B,l,q)}(a), \mathbf{g}^{(B,l,q)} \rangle \in [\frac{q}{B^l}]_{\text{sym}}$. $G^{(B,l,q)}(\cdot)$ is applied to integer vectors in the natural way. We omit q in the superscript when it is clear from the context.

For a finite set S , $a \leftarrow S$ means a is uniformly sampled from S . For a distribution χ , $a \leftarrow \chi$ means a is sampled according to χ . The variance of a random variable a is denoted as $\text{Var}(a)$.

2.2 LWE and RLWE Encryption

LWE encryption of a plaintext $m \in \mathbb{Z}$ produces a ciphertext $c = (\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + m + e) \in \mathbb{Z}_q^{n+1}$, where $\mathbf{a}_i \leftarrow \mathbb{Z}_q$, $\mathbf{s} \in \mathbb{Z}^n$ is the secret key, e is the encryption noise, and n, q are the LWE dimension and ciphertext modulus. We instantiate LWE with dense binary secrets and Gaussian noise, i.e., $\mathbf{s}_i \leftarrow \{0, 1\}$, $e \leftarrow N(0, \sigma_{\text{LWE}}^2)$. $\text{LWE}_{\mathbf{s},q}(m; e)$ represents an LWE ciphertext modulo q with secret $\mathbf{s}$, message m and noise e , where $\mathbf{s}, q, e$ are sometimes omitted. For $c = (\mathbf{a}, b) = \text{LWE}_{\mathbf{s},q}(m; e)$, its phase is defined as $\psi(c) = [b - \langle \mathbf{a}, \mathbf{s} \rangle]_q = [m + e]_q^{\text{sym}}$, and its raw phase is defined as $\psi^*(c) = b - \langle \mathbf{a}, \mathbf{s} \rangle \in \mathbb{Z}$, which is $\psi(c)$ without modular reduction.

An Lev ciphertext encrypting m is a list of LWE ciphertexts encrypting the entries of $m \cdot \mathbf{g}^{(B,l,q)}$, i.e., $\text{Lev}_{\mathbf{s},q}^{(B,l,q)}(m) = (\text{LWE}_{\mathbf{s},q}(m \cdot \frac{q}{B^l}), \dots, \text{LWE}_{\mathbf{s},q}(m \cdot \frac{q}{B}))$.

RLWE encryption of a plaintext $m \in R$ produces $C = (A, B = AS + E) \in R_Q^2$, with $S \in R$ as the secret, E as the noise, and Q as the ciphertext modulus. Still, $S[i] \leftarrow \{0, 1\}$ and $E[i] \leftarrow N(0, \sigma_{\text{RLWE}}^2)$. $\psi(\cdot)$, $\text{RLWE}(\cdot)$, and $\text{RLev}(\cdot)$ are defined similarly to the LWE case.

For $A \in R_q$, let $\vec{A} = \begin{pmatrix} A[0] & -A[N-1] & \dots & -A[1] \\ A[1] & A[0] & \dots & -A[2] \\ \vdots & \ddots & \ddots & \vdots \\ A[N-1] & A[N-2] & \dots & A[0] \end{pmatrix} \in \mathbb{Z}_q^{N \times N}$. The RLWE

ciphertext $C = (A, B = AS + M + E)$ can be represented in the linear format as $\vec{B} = \vec{A}\vec{S} + \vec{M} + \vec{E}$. The i -th row (column) of $\vec{A}$ are denoted as $\vec{A}[i, :]$ ($\vec{A}[:, i]$).

2.3 TFHE Overview

TFHE scheme utilizes RLWE to bootstrap LWE ciphertexts and evaluate an LUT on the LWE plaintext simultaneously. Let $\mathbf{s} \in \mathbb{Z}^n$ be the LWE secret, and $S \in R$ be the RLWE secret. LWE and RLWE have the same ciphertext modulus q . TFHE bootstrapping consists of the following subroutines.

- **ModSwitch**($c = (\mathbf{a}, b) = \text{LWE}_{\mathbf{s}, q}(m; e), q'$):
Output $c' = (\lfloor \frac{q'}{q} \mathbf{a} \rfloor, \lfloor \frac{q'}{q} b \rfloor) = \text{LWE}_{\mathbf{s}, q'}(\frac{q'}{q} m; \frac{q'}{q} e + e_{\text{ms}})$ with e_{ms} as the modulus-switching noise.
- **BlindRotate**($c = \text{LWE}_{\mathbf{s}, q}(m'; e), C = \text{RLWE}_S(\text{TV}; E), \text{BRK}^{(B_{\text{br}}, l_{\text{br}})})$:
Output $\text{RLWE}_S(\text{TV} \cdot X^{-\psi^*(c)}; E + E_{\text{br}})$ with E_{br} as the blind rotation noise.
Here $\text{BRK}^{(B_{\text{br}}, l_{\text{br}})} = ((\text{RLev}_S^{(B_{\text{br}}, l_{\text{br}})}(X^{s_i}), \text{RLev}_S^{(B_{\text{br}}, l_{\text{br}})}(S \cdot X^{s_i})))_{i \in [n]}$ is the key material for blind rotation.
- **KeySwitch**($c = \text{LWE}_{\mathbf{s}, q}(m'; e), \text{KSK}_{\mathbf{s} \rightarrow \mathbf{s}'}^{(B_{\text{ks}}, l_{\text{ks}})})$:
Output $\text{LWE}_{\mathbf{s}', q}(m'; e + e_{\text{ks}})$, with e_{ks} as the key-switching noise.
 $\text{KSK}_{\mathbf{s} \rightarrow \mathbf{s}'}^{(B_{\text{ks}}, l_{\text{ks}})} = (\text{Lev}_{\mathbf{s}', q}^{(B_{\text{ks}}, l_{\text{ks}})}(\mathbf{s}[i]))_{i \in [n]}$ is the key-switching keys.
- **SampleExtract**($C = (A, B) = \text{RLWE}_S(M; E), i \in [N]$):
Output $c = (\vec{A}[i, :], B[i]) = \text{LWE}_{\mathbf{s}, q}(M[i]; E[i])$.

We use Chillotti et al.'s analysis [10] for the variances of e_{ms} , E_{br} and e_{ks} .

Given $c = \text{LWE}_{\mathbf{s}, q}(\frac{q}{t} m; e)$ and a negacyclic function $f(x) : \mathbb{Z}_t \rightarrow \mathbb{Z}_t$ as inputs, TFHE programmable bootstrapping performs the following steps in order.

1. Construct $\text{TV} = X^{-N/t} \sum_{i=0}^{t/2-1} \left(\frac{q}{t} f(i) X^{2Ni/t} \cdot \sum_{j=0}^{2N/t-1} X^j \right) \in R_q$.
2. $c' = \text{ModSwitch}(c, 2N) = \text{LWE}_{\mathbf{s}, 2N}(\frac{2N}{t} m; e_{\text{in}})$, with $e_{\text{in}} = \frac{2N}{q} e + e_{\text{ms}}$.
3. $C = \text{BlindRotate}(c', \text{TV}, \text{BRK}^{(B_{\text{br}}, l_{\text{br}})}) = \text{RLWE}_{S, q}(\text{TV} \cdot X^{-\frac{2N}{t} m - e_{\text{in}}}; E_{\text{br}})$.
4. $c_1 = \text{SampleExtract}(C, 0) = \text{LWE}_{\mathbf{s}, q}(\frac{q}{t} f(m); e_{\text{br}})$.
5. Perform homomorphic additions or scalar multiplications on c_1 and other sample-extracted ciphertexts, with c_2 as the result.
6. $c_3 = \text{KeySwitch}(c_2, \text{KSK}_{\mathbf{s} \rightarrow \mathbf{s}}^{(B_{\text{ks}}, l_{\text{ks}})})$, which serves as the input of the next PBS.

2.4 Homomorphic Linear Transformations on Coefficients

The public functional key-switching [9] can homomorphically evaluate a $\mathbb{Z}$ -linear function $f : \mathbb{Z}_q^z \rightarrow R_q$ through

$$\begin{aligned}
& \underline{\text{PubFuncKS}}(f, \{c^{(i)}\}_{i \in [z]} = \{(\mathbf{a}^{(i)}, b^{(i)})\} = \{\text{LWE}_{\mathbf{s}}(m^{(i)}; e^{(i)})\}, \{\text{RLev}_S^{(B, l, q)}(\mathbf{s}_j)\}_{j \in [n]}) \\
&= \sum_{i \in [n]} \left\langle G^{(B, l, q)}(f(\{\mathbf{a}_j^{(i)}\}_{j \in [n]})), \text{RLev}_S^{(B, l, q)}(\mathbf{s}_j) \right\rangle + f(\{\mathbf{a}_j^{(i)}\}_{j \in [n]}) \\
&= \text{RLWE}_S(f(\{m_{i \in [z]}^{(i)}\}); f(\{e_{i \in [z]}^{(i)}\}) + E_{\text{pfks}}),
\end{aligned}$$

where E_{pfks} is the public functional key-switching noise.

For the special linear transformation of LWE-to-RLWE packing where each input LWE plaintext is packed into B consecutive coefficients of the RLWE plaintext, the **base-aware PackingKS** [16] moves the second factor $\sum_{k \in [B]} X^k$ in $f(\{m^{(i)}\}) = \sum_{i \in [z]} m^{(i)} X^{iB} \cdot \sum_{k \in [B]} X^k$ into the RLev ciphertexts, thus reducing the variance of E_{pfks} by B times. Also, when z is small, each polynomial-RLev product between $G^{(B,l,q)}(f(\{\mathbf{a}_j^{(i)}\}_{j \in [n]}))$ and $\text{RLev}_S^{(B,l,q)}(\mathbf{s}_j \cdot \sum_{k \in [B]} X^k)$ can be replaced by z integer-RLev products, reducing the computational cost.

When the input LWE ciphertexts for LWE-to-RLWE packing are extracted from the same RLWE ciphertext, the packing function $f : \mathbb{Z}_q^z$ can be expressed as a function $F : R_q \rightarrow R_q$. For example, $F(M) = \sum_{i \in [z]} M[i] \cdot X^{iB} \cdot \sum_{k \in [B]} X^k$ packs and duplicates the first z coefficients. Such a $\mathbb{Z}$ -linear F can be evaluated on the input RLWE ciphertext using the column method for matrix-vector multiplication [17] as

$$\begin{aligned} \text{HomLinTrans}(F, C = (A, B) = \text{RLWE}_S(M; E), \{\text{RLev}_S^{(B,l,q)}(S[i])\}_{i \in [N]}) \\ = \sum_{i \in [N]} \left\langle G^{(B,l,q)}(F(A \cdot X^i), \text{RLev}_S^{(B,l,q)}(S[i])) \right\rangle + F(B) \\ = \text{RLWE}_S(F(M); F(E) + E_{\text{pfks}}). \end{aligned}$$

The technique of base-aware PackingKS can still be applied to reduce the noise variance.

3 Core Components for Meta-PBS

3.1 Homomorphic Approximate Euclidean Division

Homomorphic approximate Euclidean division can be understood as a modulus-switching with remainder. It is used to derive the LWE ciphertexts used in blind rotations in Meta-PBS.

Definition 1. For an LWE ciphertext $c = (\mathbf{a}, b) \in \mathbb{Z}^{n+1}$ and a target quotient modulus $q' \mid q$, define $\text{HomDivRem}(c, q') = (c_{\text{quo}}, c_{\text{rem}})$, where

$$\begin{aligned} c_{\text{rem}} &= ([\mathbf{a}]_{\frac{q}{q'}}^{\text{sym}}, [b]_{\frac{q}{q'}}^{\text{sym}}) \in \mathbb{Z}^{n+1}, \\ c_{\text{quo}} &= \frac{q'}{q}(c - c_{\text{rem}}) \in \mathbb{Z}^{n+1}. \end{aligned}$$

Remark. The original version of homomorphic Euclidean division [8] sets $c_{\text{quo}} = \text{ModSwitch}(c, q') \in \mathbb{Z}_{q'}^{n+1}$, while our definition differs from it by not reducing c_{quo} modulo q' . This modification helps to preserve the information of $\psi^*(c)$ as

$$\psi^*(c) = \frac{q}{q'} \psi^*(c_{\text{quo}}) + \psi^*(c_{\text{rem}}).$$

Lemma 1. [10] For an LWE ciphertext c with a secret $\mathbf{s} \leftarrow \{0, 1\}^n$ modulo a sufficiently large q ,

$$\Pr[|\psi^*(c)/q| \geq \delta] = \text{erfc}\left(\delta / \sqrt{2 \cdot (n/24 + 1/12)}\right).$$

3.2 Conversion Between R_q and Laurent Polynomials

We will define the `truncRepeat` map from the perspective of Laurent polynomials, whose relationship with R_q elements is described in this subsection. Lemma 3 is proved in Supplementary Materials A.

Definition 2. For $M \in R_q$ and $S \subset \mathbb{Z}$, define

$$\text{Trunc}(M, S) = \sum_{i \in S} M[[i]_N] \cdot X^{[i]_N} \in R_q.$$

Definition 3. For $M \in R_q$, define $\text{Lift}(M)$ as the unique Laurent polynomial $M' \in \mathbb{Z}_q[X, X^{-1}]$ such that

$$M' \equiv M \bmod X^N + 1, \text{ and } \text{supp}(M') \subseteq [N]_{\text{sym}}.$$

Definition 4. For $M \in \mathbb{Z}_q[X, X^{-1}]$, define

$$\text{Embed}(M) = M \bmod X^N + 1 \in R_q.$$

Lemma 2. For a fixed $S \subset \mathbb{Z}$, $\text{Trunc}(\cdot, S)$, $\text{Embed}(\cdot)$, and $\text{Lift}(\cdot)$ are $\mathbb{Z}$ -linear. Also, for $S = S_1 \cup S_2$ with $S_1 \cap S_2 = \emptyset$, $\text{Trunc}(\cdot, S) = \text{Trunc}(\cdot, S_1) + \text{Trunc}(\cdot, S_2)$.

Lemma 3. For $i \in \mathbb{Z}$, multiplication by X^i commutes with Trunc , Lift , and Embed under certain conditions.

- $\text{Trunc}(m, S) \cdot X^i = \text{Trunc}(m \cdot X^i, S + i)$, for $m \in R_q$ and $S \subset \mathbb{Z}$.
- $\text{Embed}(m \cdot X^i) = \text{Embed}(m) \cdot X^i$, for $m \in \mathbb{Z}[X, X^{-1}]$.
- $\text{Lift}(m \cdot X^i) = \text{Lift}(m) \cdot X^i$ for $m \in R_q$ with $[\text{supp}(m)]_N^{\text{sym}} + i \subseteq [N]_{\text{sym}}$.

3.3 The `TruncRepeat` Map

The key operation in Meta-PBS is the `TruncRepeat` map, which homomorphically truncates a polynomial by discarding some terms and duplicates the remaining ones. We denote the plaintext version of `TruncRepeat` as `truncRepeat`.

Definition 5. For $M \in R_q$, $B \in \mathbb{Z}_{>0}$, define

$$\begin{aligned} \text{truncPad}(M, \llbracket a, b \rrbracket, B) &= \text{Embed}(\pi_B(\text{Lift}(\text{Trunc}(M, \llbracket a, b \rrbracket)))) \in R_q, \\ \text{truncRepeat}(M, \llbracket a, b \rrbracket, B) &= \text{truncPad}(M, \llbracket a, b \rrbracket, B) \cdot \sum_{i \in [B]_{\text{sym}}} X^i \in R_q. \end{aligned}$$

Definition 6. $M \in R_q$ is called r -redundant in $v \in \mathbb{Z}_q$ if

$$\forall i \in [r]_{\text{sym}}, (M \cdot X^{-i})[0] = v.$$

Recall that Meta-PBS increases the LUT redundancy iteratively by applying `truncRepeat`. This functionality of `truncRepeat` is stated in Theorem 1.

Theorem 1. For $a, b, i \in \mathbb{Z}$ and $B \in \mathbb{Z}_{\geq 2}$, if $\max(|a|, |b|) \cdot B < N$, $(b - a + 1)B \leq N$, M is B_0 -redundant in $v \in \mathbb{Z}_q$, and $[B_0]_{\text{sym}} \subseteq \llbracket a, b \rrbracket - i \subseteq [N]_{\text{sym}}$,

$$\text{truncRepeat}(M \cdot X^i, \llbracket a, b \rrbracket, B) \cdot X^{-iB + \Delta_{B_0, B}} \text{ is } BB_0\text{-redundant in } v,$$

where $\Delta_{B_0, B} = \min[BB_0]_{\text{sym}} - B \cdot \min[B_0]_{\text{sym}} - \min[B]_{\text{sym}}$.

Roughly speaking, the theorem states that for a polynomial M that is B_0 -redundant in v , $\text{truncRepeat}(M \cdot X^i, \llbracket a, b \rrbracket, B) \cdot X^{-iB}$ is almost BB_0 -redundant in v . The sizes of the parameters a, b, B , and i need to be small. Otherwise, either the duplicated coefficients will overlap with each other, or the X^i terms will fail to propagate outside of truncRepeat as some coefficients of $M \cdot X^i$ may be too far away from the constant term. The proof of Theorem 1 relies on the following lemmas, whose proofs will be included in Supplementary Materials A.

Lemma 4. For $i, a, b \in \mathbb{Z}$ and $M \in R_q$ with $[\llbracket a, b \rrbracket_N^{\text{sym}} - i \subseteq [N]_{\text{sym}}$,

$$\text{truncPad}(M \cdot X^i, \llbracket a, b \rrbracket, B) = \text{truncPad}(M, \llbracket a - i, b - i \rrbracket, B) \cdot X^{iB}.$$

Lemma 5. For $a, b \in \mathbb{Z}$ with $\max(|a|, |b|) \cdot B < N$,

$$\text{truncPad}(m, \llbracket a, b \rrbracket, B) = \sum_{j \in \llbracket a, b \rrbracket_{\geq 0}} m[j] \cdot X^{jB} + \sum_{j \in \llbracket a, b \rrbracket_{< 0}} m[N + j] \cdot X^{N + jB}.$$

Lemma 6. For $a, b \in \mathbb{Z}$ and $B \in \mathbb{Z}_{\geq 2}$ with $\max(|a|, |b|) \cdot B < N$ and $(b - a + 1)B \leq N$, the following holds for any $j \in \llbracket a, b \rrbracket$,

$$\text{truncRepeat}(M, \llbracket a, b \rrbracket, B) \cdot X^{-jB} \text{ is } B\text{-redundant in } (M \cdot X^{-j})[0].$$

Proof (Theorem 1). Let $F = \text{truncRepeat}(M \cdot X^i, \llbracket a, b \rrbracket, B) \cdot X^{-iB}$. From Lemma 6, for $j \in [B_0]_{\text{sym}} \subseteq \llbracket a, b \rrbracket - i$, $F \cdot X^{-jB}$ is B -redundant in $(M \cdot X^{-j})[0]$, which equals v since M is B_0 -redundant in v . Thus, for $k \in [B]_{\text{sym}}$, $F \cdot X^{-jB - k} = v$. Then $F \cdot X^{\Delta_{B_0, B} - g} = v$ for $g \in \Delta_{B_0, B} + \text{supp}(jB + k) = [BB_0]_{\text{sym}}$. $\square$

4 Meta-PBS: Programmable Bootstrapping through Increasing the Redundancy-to-Noise Ratio Iteratively

4.1 The Basic Framework

For an input LWE message $m \in \mathbb{Z}_t$ and a negacyclic function f over $\mathbb{Z}_t$, Meta-PBS iteratively increases the redundancy of $f(m)$ in the blind-rotated test vector. The iteration ends when the redundancy is large enough so that the perturbation on the exponent does affect the result of SampleExtract . Parameters in Table 1 characterize the change in redundancy and perturbation during iterations. The full algorithm of Meta-FBS is given in Algorithm 1.

The four conditions in Theorem 2 can be intuitively understood as follows.

1. δ_i is the high-probability upper bound of e_i , the perturbation on the exponent after the i -th blind rotation.

Algorithm 1 Meta-PBS

Require: Meta-PBS parameters from Table 1

Require: A negacyclic function f over $\mathbb{Z}_t$

Require: Input LWE ciphertext $c = \text{LWE}_S(\frac{q}{t}m; e)$ with $|e| < \frac{q}{2t} \cdot 2^{-c_{\text{meta}}}$

Require: Blind rotation keys $\text{BRK}^{(B_{\text{br}}, l_{\text{br}})}$

Ensure: $c_{\text{out}} = \text{LWE}_S(\frac{q}{t}f(m))$

- 1: $\text{TV} \leftarrow X^{-N/t} \sum_{i=0}^{t/2-1} \left(\frac{q}{t}f(i)X^{2Ni/t} \cdot \sum_{j=0}^{2N/t-1} X^j \right)$
 - 2: $c_{\text{quo},0}, c_{\text{rem},0} \leftarrow \text{HomDivRem}(c, 2N)$
 - 3: $C_0 \leftarrow \text{BlindRotate}(c_{\text{quo},0}, \text{TV}, \text{BRK}^{(B_{\text{br}}, l_{\text{br}})})$
 - 4: **for** $i \in [K]$ **do**
 - 5: $C'_i \leftarrow \text{TruncRepeat}(C_i, \llbracket -T_i, T_i \rrbracket, \beta_i) \cdot X^{\Delta_{r_i, \beta_i}}$
 - 6: $c_{\text{quo},i+1}, c_{\text{rem},i+1} \leftarrow \text{HomDivRem}(c_{\text{rem},i}, \beta_i)$
 - 7: $C_{i+1} \leftarrow \text{BlindRotate}(c_{\text{quo},i+1}, C'_i, \text{BRK}^{(B_{\text{br}}, l_{\text{br}})})$
 - 8: **end for**
 - 9: $c_{\text{out}} \leftarrow \text{SampleExtract}(C_K, 0)$
-

2. The truncation bound T_i should be large enough to keep all the coefficients equal to $\frac{q}{t}f(m)$, and B should be small enough so that the duplicated coefficients do not overlap in R_q .
3. The redundancy grows at most β_i times after the i -th `truncRepeat`, and the initial redundancy is small enough so that the LUT fits into R_q .
4. The final redundancy is large enough so that `SampleExtract` outputs an encryption of $\frac{q}{t}f(m)$.

Theorem 2. *Algorithm 1 outputs $\text{LWE}_{S,q}(\frac{q}{t}f(m))$ with a failure probability no greater than $(K+1) \cdot p_{\text{fail}}$ under the following conditions, where p_{fail} is the failure probability of the noise bound δ_i in each iteration (as will be explained in the proof).*

1. $\forall i \in [K+1], \delta_i \geq \lceil \text{erfc}^{-1}(p_{\text{fail}}) \cdot \sqrt{2 \cdot (n/24 + 1/12)} \rceil + \alpha_i \cdot 2^{-1-c_{\text{meta}}}$.
2. $\forall i \in [K], (2T_i + 1) \cdot B_i \leq N, T_i \geq \delta_i + \lfloor \frac{r_i}{2} \rfloor$.
3. $r_0 \leq \frac{2N}{t}$ and $\forall i \in [K], r_{i+1} \leq r_i \cdot B_i$.
4. $r_K \geq 2\delta_K + 1$.

Proof. For $i \in [K+1]$, let $\bar{\beta}_i = \prod_{j \in [i]} \beta_j$ and $\Delta_i = \frac{q}{2N} \cdot \bar{\beta}_i^{-1} = \frac{q}{t} \alpha_i^{-1}$. Using Definition 1, we obtain

$$\psi(c_{\text{quo},0}) = \frac{2N}{t}m + \frac{e - \psi^*(c_{\text{rem},0})}{\Delta_0},$$

$$\Delta_i \cdot \psi^*(c_{\text{quo},i}) + \psi^*(c_{\text{rem},i}) = \psi^*(c_{\text{rem},i-1}), \text{ for } i \in [K+1].$$

Suppose $C_i = \text{RLWE}_S(M_i)$ for $i \in [K+1]$, we claim that

$$M_i = \bar{M}_i \cdot X^{-e_i}, \text{ where } \bar{M}_i \text{ is } r_i\text{-redundant in } \frac{q}{t}f(m) \text{ and } e_i = \frac{e - \psi^*(c_{\text{rem},i})}{\Delta_i}.$$

Table 1: Parameters for Meta-PBS. The index i starts from 0.

Name	Description
K	A non-negative integer. The number of iterations.
r	A length $K + 1$ list of positive integers. The desired LUT entry after the i -th blind rotation has a redundancy of r_i .
β	A length K list of positive integers with $\beta_i \geq 2$. The value of β in the i -th TruncRepeat is β_i .
c_{meta}	A positive real number. The input LWE ciphertext has a capacity of c_{meta} bits.
α	A length $K + 1$ list of positive integers. After the i -th blind rotation, the noise in the input LWE ciphertext is amplified by a factor of α_i . $\alpha_i = (\text{Largest possible value of } r_0) \cdot \prod_{j \in [i]} \beta_j$, which is $\frac{2N}{t} \cdot \prod_{j \in [i]} \beta_j$ for negacyclic PBS.
δ	A length $K + 1$ list of positive integers. After the i -th blind rotation, the perturbation over the exponent is bounded by δ_i .
T	A length K list of positive integers. The i -th TruncRepeat truncates to $\llbracket -T_i, T_i \rrbracket$.

According to the definition of blind rotation, $M_0 = \text{TV} \cdot X^{-\psi(c_{\text{quo},0}) - \frac{2N}{t}m}$. Let $\bar{M}_0 = \text{TV} \cdot X^{-\frac{2N}{t}m}$ and $e_0 = \psi(c_{\text{quo},0}) - \frac{2N}{t}m$, $\bar{M}_0$ is $\frac{2N}{t}$ -redundant in $\frac{q}{t}f(m)$ by design and $e_0 = \frac{e - \psi^*(c_{\text{rem},i})}{\Delta_i}$ by Definition 1.

Condition 1 ensures that $\Pr[|e_i| > \delta_i] \leq p_{\text{fail}}$ because $|\frac{e}{\Delta_i}| \leq \alpha_i \cdot 2^{-1-c_{\text{meta}}}$ and $\Pr[|\frac{\psi^*(c_{\text{rem},i})}{\Delta_i}| > \text{erfc}^{-1}(p_{\text{fail}}) \cdot \sqrt{2 \cdot (n/24 + 1/12)}] < p_{\text{fail}}$.

Assume that the claim holds for $i - 1$. For $i \in \llbracket 1, K \rrbracket$,

- Suppose C'_{i-1} encrypts M'_{i-1} , then $M'_{i-1} \cdot X^{e_{i-1}B_{i-1}}$ is $r_{i-1}B_{i-1}$ -redundant in $\frac{q}{t}f(m)$, due to $|e_{i-1}| \leq \delta_{i-1}$, Condition 2, Theorem 1³, and the induction hypothesis. Condition 3 guarantees that the polynomial is also r_i -redundant.
- $M_i = M'_{i-1} \cdot X^{-\psi^*(c_{\text{quo},i})} = (M'_{i-1} \cdot X^{e_{i-1}\beta_{i-1}}) \cdot X^{-e_{i-1}\beta_{i-1} - \psi^*(c_{\text{quo},i})}$. The induction hypothesis imply $e_{i-1}\beta_{i-1} + \psi^*(c_{\text{quo},i}) = (e - \psi^*(c_{\text{rem},i-1}) + \Delta_i\psi^*(c_{\text{quo},i}))/\Delta_i = (e - \psi^*(c_{\text{rem},i}))/\Delta_i$.
- Setting $\bar{M}_i = M'_{i-1} \cdot X^{e_{i-1}\beta_{i-1}}$ and $e_i = \frac{e - \psi^*(c_{\text{rem},i})}{\Delta_i}$ finishes the induction.

Finally, combining the claim with Condition 4 produces $c_{\text{out}} = \text{LWE}_{\mathcal{S}}(\frac{q}{t}f(m))$. The overall failure probability is bounded by $(K + 1) \cdot p_{\text{fail}}$ using the union bound. $\square$

The noise growth of Meta-PBS will be discussed in the end of Section 4.3, after we have settled the realization of TruncRepeat.

³ $\llbracket -T_{i-1}, T_{i-1} \rrbracket + e_{i-1} \subseteq [N]_{\text{sym}}$ is ensured by $\delta_{i-1} < T_{i-1} < \frac{N}{4}$.

4.2 Obtaining Lower Noise Bounds for Gaussian Input Noise

Usually, the noise e in the input LWE ciphertext of Algorithm 1 can be modelled as a centered Gaussian variable. In this case, we could achieve smaller values of δ_i by viewing each e_i as a Gaussian variable.

Corollary 1. *If we replace the requirement for the input LWE noise e in Algorithm 1 as $\text{Var}(e) \leq V_{\text{in}, \text{Cmeta}} := (\frac{q}{2t} \cdot 2^{-c_{\text{meta}}} / (\text{erfc}^{-1}(p_{\text{fail}}) \cdot \sqrt{2}))^2$, Algorithm 1 outputs $\text{LWE}_S(\frac{q}{t}m)$ with a failure probability no more than $(K+1) \cdot p_{\text{fail}}$ under Condition 2,3,4, and a modified version of Condition 1*

$$\forall i \in [K+1], \delta_i \geq \left\lceil \text{erfc}^{-1}(p_{\text{fail}}) \cdot \sqrt{2 \cdot (n/24 + 1/12) + 2 \left(\frac{\alpha_i t}{q}\right)^2 \cdot V_{\text{in}}} \right\rceil.$$

Proof. Following the proof of Theorem 2, the perturbation after the i -th blind rotation, e_i , is exactly the noise in $\text{ModSwitch}(c, 2N\beta)$ and has a variance of

$$\text{Var}(e_i) = \text{Var}\left(\frac{e}{\Delta_i}\right) + \text{Var}\left(\frac{\psi^*(c_{\text{rem}, i})}{\Delta_i}\right) = \left(\frac{\alpha_i t}{q}\right)^2 V_{\text{in}} + \frac{n}{24} + \frac{1}{12}. \quad \square$$

4.3 TruncRepeat with Low Memory and Time Cost

The naive approach. The $\mathbb{Z}$ -linearity of TruncPad means that it could be realized using the column method in Section 2.4. For gadget parameters B, l ,

$$\begin{aligned} & \text{TruncPad}(C = (A, B), \llbracket a, b \rrbracket, \beta) \\ &= \sum_{i \in [N]} \langle G^{(B, l, q)}(\text{truncPad}(A \cdot X^i, \llbracket a, b \rrbracket, \beta)), \text{RLev}^{(B, l, q)}(S[i]) \rangle + \text{truncPad}(B, \llbracket a, b \rrbracket, \beta) \end{aligned}$$

Equation 1 gives a toy example of the corresponding matrix-vector multiplication for $C = (A, B) = \text{RLWE}_S(M; E)$, with $N = 8$, $a = -1$, $b = 1$ and $\beta = 2$.

$$\begin{pmatrix} A[0] & -A[7] & \cdots & -A[1] \\ 0 & 0 & \cdots & 0 \\ A[1] & A[0] & \cdots & -A[2] \\ 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ A[7] & A[6] & \cdots & A[0] \\ 0 & 0 & \cdots & 0 \end{pmatrix} \cdot \begin{pmatrix} S[0] \\ S[1] \\ S[2] \\ S[3] \\ S[4] \\ S[5] \\ S[6] \\ S[7] \end{pmatrix} + \begin{pmatrix} B[0] \\ 0 \\ B[1] \\ 0 \\ 0 \\ 0 \\ B[7] \\ 0 \end{pmatrix} = \begin{pmatrix} M[0] + E[0] \\ 0 \\ M[1] + E[1] \\ 0 \\ 0 \\ 0 \\ M[7] + E[7] \\ 0 \end{pmatrix} \quad (1)$$

TruncRepeat could be realized by replacing $\text{RLev}^{(B, l_{\text{tr}}, q)}(S[i])$ in TruncPad with $\text{RLev}^{(B, l_{\text{tr}}, q)}(S[i] \cdot \sum_{k \in [\beta]_{\text{sym}}} X^k)$. This realization of $\text{TruncPad}/\text{TruncRepeat}$ has a computational cost of $N^2 \log N l_{\text{tr}} \mathbb{Z}_q$ -multiplications and a storage cost of $2N^2 l_{\text{tr}} \mathbb{Z}_q$ -integers in the key-switching key, which compares to the costs of a blind rotation. In the following, we show how to significantly reduce the noise variance, computational and storage costs of TruncRepeat .

Compact TruncRepeat from Locally Negacyclic Matrix. Recall that Lemma 6 requires $b - a + 1 \leq \lfloor \frac{N}{\beta} \rfloor$ to prevent coefficients overlapping in the output of TruncRepeat. When $b - a + 1 < \lfloor \frac{N}{\beta} \rfloor$, there are $\lfloor \frac{N}{\beta} \rfloor - (b - a + 1)$ unused but available rows in the matrix-vector multiplication of TruncPad. We could exploit these unused rows to create local negacyclicity in the LHS matrix of Equation 1, thus merging multiple polynomial-RLev multiplications into a single one.

Suppose $a \leq 0 \leq b$, $\beta \geq 2$ and $b - a + 1 < \lfloor \frac{N}{\beta} \rfloor$ (which is exactly the case in Algorithm 1). Let $\epsilon \leq \lfloor \frac{N}{\beta} \rfloor - (b - a + 1)$ and $\llbracket k, k + \epsilon \rrbracket \subseteq [N]$, we merge the consecutive $\epsilon + 1$ columns with index in $\llbracket k, k + \epsilon \rrbracket$ into

$$\begin{aligned} & \sum_{i \in \llbracket k, k + \epsilon \rrbracket} \langle G^{(B, l, q)}(\text{truncPad}(A \cdot X^i, \llbracket a - \epsilon + i - k, b + i - k \rrbracket, \beta)), \text{RLev}^{B, l}(S[i]) \rangle \\ &= \sum_{i \in \llbracket k, k + \epsilon \rrbracket} \langle G^{(B, l, q)}(\text{truncPad}(A \cdot X^k, \llbracket a - \epsilon, b \rrbracket, \beta) \cdot X^{\beta(i-k)}), \text{RLev}^{B, l}(S[i]) \rangle \\ &= \langle G^{(B, l, q)}(\text{truncPad}(A \cdot X^k, \llbracket a - \epsilon, b \rrbracket, \beta)), \text{RLev}^{B, l}(\sum_{i \in \llbracket k, k + \epsilon \rrbracket} S[i] \cdot X^{\beta(i-k)}) \rangle. \end{aligned}$$

Lemma 4 applies to the first equality because $0 < b + \epsilon \leq \lfloor \frac{N}{\beta} \rfloor + a - 1 < \frac{N}{2}$ and $0 > a - \epsilon \geq b + 1 - \lfloor \frac{N}{\beta} \rfloor > -\frac{N}{2}$.

Assuming $(\epsilon + 1) \mid N$, the full formula for the modified TruncPad, which we call $\text{TruncPad}^*(C, \llbracket a, b \rrbracket, \beta, \epsilon)$, becomes

$$\sum_{j \in [N/(\epsilon+1)]} \langle G^{(B, l, q)}(\text{truncPad}(A \cdot X^{j\epsilon}, \llbracket a - \epsilon, b \rrbracket, \beta)), \text{RLev}^{B, l}(\sum_{i \in \llbracket j\epsilon, (j+1)\epsilon \rrbracket} S[i] \cdot X^{\beta(i-j\epsilon)}) \rangle.$$

A toy example of TruncPad^* based on Equation 1 with $\epsilon = 1$ is given in Equation 2. The matrix entries that match Equation 1 are highlighted in red, while entries marked by $*$ and $\dots$ are constructed through negacyclicity.

$$\begin{pmatrix} \textcolor{red}{A[0]} * \textcolor{red}{-A[7]} \dots \\ 0 * 0 \dots \\ \textcolor{red}{A[1]} * \textcolor{red}{A[0]} \dots \\ 0 * 0 \dots \\ A[6] * A[1] \dots \\ 0 * 0 \dots \\ \textcolor{red}{A[7]} * \textcolor{red}{A[6]} \dots \\ 0 * 0 \dots \end{pmatrix} \cdot \begin{pmatrix} S[0] \\ 0 \\ S[1] \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} \textcolor{red}{-A[6]} * \textcolor{red}{-A[5]} \dots \\ 0 * 0 \dots \\ \textcolor{red}{-A[7]} * \textcolor{red}{-A[6]} \dots \\ 0 * 0 \dots \\ A[4] * -A[7] \dots \\ 0 * 0 \dots \\ \textcolor{red}{A[5]} * \textcolor{red}{A[4]} \dots \\ 0 * 0 \dots \end{pmatrix} \cdot \begin{pmatrix} S[2] \\ 0 \\ S[3] \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \dots \quad (2)$$

$$+ \begin{pmatrix} \textcolor{red}{-A[2]} * \textcolor{red}{-A[1]} \dots \\ 0 * 0 \dots \\ \textcolor{red}{-A[3]} * \textcolor{red}{-A[2]} \dots \\ 0 * 0 \dots \\ A[0] * -A[3] \dots \\ 0 * 0 \dots \\ \textcolor{red}{A[1]} * \textcolor{red}{A[0]} \dots \\ 0 * 0 \dots \end{pmatrix} \cdot \begin{pmatrix} S[6] \\ 0 \\ S[7] \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} B[0] \\ 0 \\ B[1] \\ 0 \\ 0 \\ 0 \\ B[7] \\ 0 \end{pmatrix} = \begin{pmatrix} M[0] + E[0] \\ 0 \\ M[1] + E[1] \\ 0 \\ \mathbf{Garbage} \\ 0 \\ M[7] + E[7] \\ 0 \end{pmatrix} \quad (3)$$

The formula for TruncRepeat^* is obtained by replacing the RLev ciphertexts in TruncPad^* with $\text{RLev}^{B,l}(\sum_{i \in \llbracket j\epsilon, (j+1)\epsilon \rrbracket} S[i] \cdot X^{\beta(i-j\epsilon)} \cdot \sum_{k \in [\beta]_{\text{sym}}} X^k)$.

Using Lemma 5, TruncRepeat^* produces garbage coefficients in rows with index in $[\bigcup_{u \in \llbracket a-\epsilon, a-1 \rrbracket \cup \llbracket b+1, b+\epsilon \rrbracket} (u\beta + [\beta]_{\text{sym}})]_N$. However, following a similar reasoning as in the proof of Lemma 6, we know TruncRepeat^* agrees with TruncRepeat on the ‘useful’ coefficients, i.e.,

$$\begin{aligned} & \text{Trunc}(\text{TruncRepeat}^*(C, \llbracket a, b \rrbracket, \beta, \epsilon), \text{supp}(\text{TruncRepeat}(C, \llbracket a, b \rrbracket, \beta))) \\ &= \text{Trunc}(\text{TruncRepeat}^*(C, \llbracket a, b \rrbracket, \beta, \epsilon), [\bigcup_{u \in \llbracket a, b \rrbracket} (u\beta + [\beta]_{\text{sym}})]_N) \\ &= \text{TruncRepeat}(C, \llbracket a, b \rrbracket, \beta). \end{aligned}$$

This leads us to the following corollary.

Corollary 2. *For $i \in [K]$, if the i -th $\text{TruncRepeat}(\dots)$ in Algorithm 1 is replaced by $\text{TruncRepeat}^*(\dots, \epsilon[i])$, the algorithm outputs $\text{LWE}_{\mathbf{S}, q(\frac{q}{t}m)}$ with a failure probability no more than $(K+1) \cdot p_{\text{fail}}$ under Condition 1, 3, 4, and replacing Condition 2 with*

$$\forall i \in [K], (2T_i + 1 + \epsilon_i) \cdot \beta_i \leq N, T_i \geq \delta_i + \lfloor \frac{r_i}{2} \rfloor.$$

The noise growth of $\text{TruncRepeat}/\text{TruncRepeat}^*$ is given in Lemma 7.

Lemma 7. *For $C = \text{RLWE}_S(M; E)$, $a, b \in \mathbb{Z}$, $\beta \in \mathbb{Z}_{>0}$ and $\epsilon \in \mathbb{Z}_{\geq 0}$, let $C' = \text{TruncRepeat}^*(C, \llbracket a, b \rrbracket, \beta, \epsilon) = \text{RLWE}_S(\text{truncRepeat}^*(M, \llbracket a, b \rrbracket, \beta, \epsilon); E')$, $B_{\text{tr}}, l_{\text{tr}}$ be the gadget parameters for TruncRepeat^* , and σ_{trk} be the standard deviation in RLev encryptions. If $\text{Var}(E[i]) = \sigma^2$ for $i \in \llbracket a, b \rrbracket_N$, $\max(|a|, |b|)\beta < N$ and $(b-a+1+\epsilon)\beta \leq N$, then for $i \in [\text{supp}(\text{truncRepeat}(M, \llbracket a, b \rrbracket, \beta))]_N$,*

$$\begin{aligned} V_{\text{tr}, \epsilon} &:= \text{Var}(E'[i]) - \sigma^2 \\ &= \frac{N}{2} \left(\frac{q^2}{12B_{\text{tr}}^{2l_{\text{tr}}}} - \frac{1}{12} \right) + \frac{N}{16} + ((b-a) \lceil \frac{N}{\epsilon+1} \rceil + N) l_{\text{tr}} (B_{\text{tr}}^2 + 2) \left(\frac{\sigma_{\text{trk}}^2}{12} + 2^{19.4} \cdot N \right). \end{aligned}$$

For $i \in [N] \setminus [\text{supp}(\text{truncRepeat}(M, \llbracket a, b \rrbracket, \beta))]_N$,

$$V_{\text{tr}, \text{empty}, \epsilon} := \text{Var}(E'[i]) = ((b-a) \lceil \frac{N}{\epsilon+1} \rceil + N) l_{\text{tr}} (B_{\text{tr}}^2 + 2) \left(\frac{\sigma_{\text{trk}}^2}{12} + 2^{19.4} \cdot N \right).$$

Proof (sketch). Following the analysis of PackingKS in [10], for $i \in [\text{supp}(\text{truncRepeat}(M, \llbracket a, b \rrbracket, \beta))]_N$, $E'[i]$ is broken down into the following parts.

The first component is equal to one of the coefficients from E , according the definition of truncRepeat , which has a variance of σ^2 .

The second component is the inner product between the coefficients of S and a vector of decomposition residues, which is the same to that in PackingKS , having a variance of $\frac{N}{2} \left(\frac{q^2}{12B_{\text{tr}}^{2l_{\text{tr}}}} - \frac{1}{12} \right) + \frac{N}{16}$.

The third component comes from the encryption noise in RLev ciphertexts. When $(\epsilon+1) \mid N$, $G^{(B_{\text{tr}}, l_{\text{tr}}, q)}$ acts on $\frac{N}{\epsilon+1}$ polynomials, each with a support size

of $b - a + 1 + \epsilon$. The final noise polynomial is the inner product between $\frac{N}{\epsilon+1}l_{\text{tr}}$ decomposed polynomials and a vector of encryption noises in RLev ciphertexts, and the final component is a coefficient in this polynomial. Thus, it has a variance of $(b - a + 1 + \epsilon) \frac{N}{\epsilon+1} l_{\text{tr}} \frac{B_{\text{tr}}^2 + 2}{12} \sigma_{\text{trk}}^2 = ((b - a) \frac{N}{\epsilon+1} + N) l_{\text{tr}} \frac{B_{\text{tr}}^2 + 2}{12} \sigma_{\text{trk}}^2$.

Now let $\Gamma = \lceil \frac{N}{\epsilon+1} \rceil$ be the number of polynomials to be gadget decomposed. Suppose $\frac{N}{\Gamma} \leq \epsilon + 1 < \frac{N}{\Gamma-1}$, there are $\Gamma - 1$ polynomials with support size $b - a + 1 + \epsilon$ and a remaining one with support size $b - a + 1 + N - (\Gamma - 1)(\epsilon + 1) - 1$. Thus, the variance is $((b - a + 1 + \epsilon)(\Gamma - 1) + b - a + 1 + N - (\Gamma - 1)(\epsilon + 1) - 1) l_{\text{tr}} \frac{B_{\text{tr}}^2 + 2}{12} \sigma_{\text{trk}}^2 = ((b - a)\Gamma + N) l_{\text{tr}} \frac{B_{\text{tr}}^2 + 2}{12} \sigma_{\text{trk}}^2$.

We also need to take the FFT noise into account, which, following the heuristic formula for FFT noise variance in blind rotation [3], is estimated as $((b - a)\Gamma + N) l_{\text{tr}} B_{\text{tr}}^2 N \cdot 2^{19.4}$.

The second conclusion holds because the third component (with FFT noise) is the only noise component in that case. $\square$

Comparing with other LWE-to-RLWE packing methods. We briefly compare TruncRepeat* with existing packing methods concerning their computational cost, evaluation key size, noise growth, and functionality to compute $\text{truncRepeat}(\cdot, \llbracket -T_i, T_i \rrbracket, \beta_i)$ homomorphically.

PubFuncKS [9] and *base-aware PackingKS* [16]. As discussed in Section 2.4, base-aware PackingKS (with polynomial multiplication) is equivalent to the column method version of TruncRepeat, i.e., TruncRepeat* with $\epsilon = 0$. The third noise component of TruncRepeat is roughly $\epsilon + 1$ times that of TruncRepeat*. Also, the computational and storage costs of TruncRepeat are roughly $\epsilon + 1$ times that of TruncRepeat*. Thus, TruncRepeat* outperforms base-aware PackingKS (and the original PubFuncKS) in noise growth, computational cost, and evaluation key size.

Automorphism-based packing [7, 20]. The main advantages of automorphism-based packing methods are the small evaluation key size (i.e., only $\log N$ keys) and a small additive noise growth equal to $\log N$ key-switchings. However, they are designed for power-of-two β and only compute $\text{truncPad}(\cdot, \llbracket -T_i, T_i \rrbracket, \beta_i)$ instead of truncRepeat , leading to additional problems.

For a power-of-two β_i , these methods cost $2T_i + 1 + \log(N/\beta_i)$ automorphisms and have an additive noise growth of $\log N$ key-switchings. The resulting ciphertext needs to be multiplied with $\sum_{j \in [\beta_i]} X^j$ to produce the truncRepeat result, amplifying the noise variance by β_i .

However, restricting β_i to powers of two will greatly limit the search space of the parameters, potentially increasing the number of required iterations. For non-power-of-two β_i , a possible way to realize truncRepeat with automorphism-based packing methods is to pack all the $\beta_i(2T_i + 1)$ coefficients of the truncRepeat result with a packing base of one. This costs no less than $\beta_i(2T_i + 1)$ automorphisms and introduces a noise equal to $\log_N$ key-switchings.

In summary, for parameters adopted in our experiments, automorphism-based packing methods have a much smaller evaluation key size, a somewhat lower noise variance, and a higher computational cost compared to TruncRepeat*. More details can be found in Supplementary Materials B.

Meta-PBS with TruncRepeat*. Finally, we are able to describe the noise growth of Meta-PBS.

Theorem 3. *The output ciphertext of Algorithm 1 has a noise variance of*

$$V_{\text{Meta-PBS}} = (K + 1) \cdot V_{\text{br}} + \sum_{i \in [K]} V_{\text{tr}, \epsilon_i}.$$

Proof. The conclusion holds because both BlindRotate and TruncRepeat* incur an additive noise growth on the noise-free TV. $\square$

Homomorphic arithmetic, like additions and constant multiplications, can be carried out on the output of Meta-PBS, before key-switching brings the ciphertext under secret s . Thus, we define c_{Hom} , the maximum capacity bits for post-bootstrap arithmetic, as the maximum non-negative real number satisfying

$$2^{2c_{\text{Hom}}} \cdot V_{\text{Meta-PBS}} + V_{\text{ks}} \leq V_{\text{in}, c_{\text{meta}}}.$$

4.4 Asymptotic Analysis

For an RLWE dimension N and LWE dimension $n = O(N)$, evaluating a negacyclic function with $t = 2N$ in PBS/EBS requires using a test vector of size $O(N^{1.5})$. Thus, the overall complexity would equal that of $O(\sqrt{N})$ blind rotations in R . We show that Meta-PBS could lower the complexity to $O(1)$ blind rotations in R asymptotically.

Theorem 4. *For RLWE dimension N , LWE dimension $n = O(N)$, and $|e| < \frac{q}{4N^2}$, Meta-PBS with $K = 2$ is able to evaluate a negacyclic function with $t = 2N$ asymptotically.*

Proof. Following Lemma 1, let $a\sqrt{N}$ be an asymptotic upper bound on the raw phase of any dimension n LWE ciphertext, where $a \geq 1$ is a constant related to the failure probability. Optimizations on Algorithm 1 are disabled to simplify the analysis. Following Theorem 2, we aim to find β_0 and β_1 that satisfy the following properties.

$$\begin{aligned} \beta_0(2\lceil a\sqrt{N} + \frac{2N}{q}e \rceil + 1) &\leq N, \\ \beta_1(2\lceil a\sqrt{N} + \lfloor \beta_0/2 \rfloor + \frac{2N}{q}e\beta_0 \rceil + 1) &\leq N, \\ \beta_0\beta_1 &\geq 2\lceil a\sqrt{N} + \frac{2N}{q}e\beta_0\beta_1 \rceil + 1. \end{aligned}$$

Obviously $\beta_0, \beta_1 < \sqrt{N}$, meaning $\frac{4N}{q}e\beta_0\beta_1 < 1$. Let $b = 2a\sqrt{N} + 5$, we have

$$\begin{aligned}\beta_0 b &\leq N, \\ \beta_1(b + \beta_0) &\leq N, \\ \beta_0 \beta_1 &\geq b.\end{aligned}$$

One can verify that every $(\beta_0, \beta_1) \in [\frac{N}{2b}, \frac{N}{b}] \times [\frac{2b^2}{N}, \frac{bN}{b^2+N}]$ satisfy the equation above. Since the region has a size of $O(\sqrt{N}) \times O(\sqrt{N})$, an integer solution for β_0, β_1 is guaranteed. $\square$

5 Advanced LUT Evaluation

In addition to regular PBS, there exist LUT evaluation methods with extended functionalities such as PBSManyLUTs and WoPPBS [10]. PBSManyLUTs can evaluate multiple negacyclic LUTs on the same LWE input in a single blind rotation, while WoPPBS supports the evaluation of non-negacyclic functions. In this section, we propose Meta-PBS versions of PBSManyLUTs and WoPPBS, which we call Meta-PBSManyLUTs and Meta-WoPPBS, respectively. The optimizations of TruncRepeat* and Gaussian input noise are adopted by default.

5.1 Meta-PBSManyLUTs

The first attempt. Before describing our method, we first review how the original PBSManyLUTs works. Given 2^ν negacyclic functions modulo $t \leq 2N \cdot 2^{-\nu}$, which we denote as f_k for $k \in [2^\nu]$, PBSManyLUTs encodes them as

$$\text{TV} = \sum_{k \in [2^\nu]} X^{-N/t+k} \sum_{i=0}^{t/2-1} \left(\frac{q}{t} f_k(i) X^{2Ni/t} \cdot \sum_{j=0}^{2N \cdot 2^{-\nu}/t-1} X^{j \cdot 2^\nu} \right).$$

It could be understood from the ring switching perspective [14]. Let TV_k be the test vector for f_k in the subring $R'_q = \mathbb{Z}_q[Y]/(Y^{N'} + 1)$ with $Y = X^{2^\nu}$ and $N' = N \cdot 2^{-\nu}$, which satisfies $\text{TV}_k = Y^{-N'/t} \sum_{i=0}^{t/2-1} \left(\frac{q}{t} f_k(i) \cdot Y^{2N'i/t} \cdot \sum_{j=0}^{2N'/t-1} Y^j \right)$. Then $\text{TV} = \sum_{k \in [2^\nu]} \text{TV}_k \cdot X^k$ is the result of inverse ring switching on TV_k 's.

Blind rotation in PBSManyLUT takes $c = \text{LWE}_s(\frac{2N \cdot 2^{-\nu}}{t}m; e) \in \mathbb{Z}_{2N \cdot 2^{-\nu}}^{n+1}$ as input, and outputs an encryption of $\text{TV} \cdot X^{-2^\nu \cdot \psi^*(c)} = \sum_{k \in [2^\nu]} \text{TV}_k \cdot Y^{-\psi(c)} \cdot X^k$. Here, each $\text{TV}_k \cdot Y^{\psi(c)}$ is the blind-rotated test vector in the subring. The constant term of each TV_k is extracted by $\text{SampleExtract}(\text{TV}, k)$.

The straightforward approach is to directly incorporate this encoding into Meta-PBS. However, TruncRepeat needs to be performed on each of the blind-rotated TV_k in variable Y to preserve the ring packing format, i.e.,

$$\sum_{k \in [2^\nu]} \text{TruncRepeat}(\text{TV}_k(Y) \cdot Y^{-\psi(c)}, \llbracket -T_i, T_i \rrbracket, \beta) \cdot X^k.$$

However, we have 2^ν times many TruncRepeat to evaluate. Even worse, each TruncRepeat cannot be replaced with the fast TruncRepeat* due to $S \notin R'_q$.

An alternative TV encoding. To avoid the performance degradation of TruncRepeat, we encode f_k differently as

$$\text{TV} = \sum_{i \in [t/2]} \sum_{k \in [2^\nu]} X^{i \cdot 2N/t + k \cdot r_0} \frac{q}{t} f_k(i) \cdot \sum_{j \in [r_0]_{\text{sym}}} X^j,$$

where $r_0 \leq \frac{2N \cdot 2^{-\nu}}{t}$ is the initial redundancy. We also add a new parameter D for Meta-PBSManyLUTs, which is a length $K + 1$ vector of positive integers satisfying $D_i = r_0 \prod_{j \in [i]} \beta_j$. Note that $\text{TV} \cdot X^{-i \cdot 2N/t - k \cdot D_0}$ is D_0 -redundant in $f_k(i)$. The full algorithm is given in Algorithm 2.

Algorithm 2 Meta-PBSManyLUTs

Require: Meta-PBS parameters from Table 1.

Require: Extra parameters ν and D

Require: 2^ν negacyclic functions $\{f_k\}_{k \in [2^\nu]}$ over $\mathbb{Z}_t$

Require: Input LWE ciphertext $c = \text{LWE}_{\mathcal{S}}(\frac{q}{t}m; e)$ with $\text{Var}(e) \leq V_{\text{in}, \text{cmeta}}$

Require: Blind rotation keys $\text{BRK}^{(B_{\text{br}}, l_{\text{br}})}$

Ensure: $c_{\text{out}, k} = \text{LWE}_{\mathcal{S}}(\frac{q}{t}f_k(m))$ for $k \in [2^\nu]$

- 1: $\text{TV} \leftarrow \sum_{i \in [t/2]} \sum_{k \in [2^\nu]} X^{i \cdot 2N \cdot 2^{-\nu}/t + k \cdot r_0} \frac{q}{t} f_k(i) \cdot \sum_{j \in [r_0]_{\text{sym}}} X^j$
 - 2: $c_{\text{quo}, 0}, c_{\text{rem}, 0} \leftarrow \text{HomDivRem}(c, 2N)$
 - 3: $C_0 \leftarrow \text{BlindRotate}(c_{\text{quo}, 0}, \text{TV}, \text{BRK}^{(B_{\text{br}}, l_{\text{br}})})$
 - 4: **for** $i \in [K]$ **do**
 - 5: $C'_i \leftarrow \text{TruncRepeat}^*(C_i, \llbracket -T_i, T_i + D_i \cdot (2^\nu - 1) \rrbracket, \beta_i, \epsilon_i) \cdot X^{\Delta_{r_i, \beta_i}}$
 - 6: $c_{\text{quo}, i+1}, c_{\text{rem}, i+1} \leftarrow \text{HomDivRem}(c_{\text{rem}, i}, \beta_i)$
 - 7: $C_{i+1} \leftarrow \text{BlindRotate}(c_{\text{quo}, i+1}, C'_i, \text{BRK}^{(B_{\text{br}}, l_{\text{br}})})$
 - 8: **end for**
 - 9: **for** $k \in [2^\nu]$ **do**
 - 10: $c_{\text{out}, k} \leftarrow \text{SampleExtract}(C_K, k \cdot D_K)$
 - 11: **end for**
-

Theorem 5. *Algorithm 2 outputs $\{\text{LWE}_{\mathcal{S}}(\frac{q}{t}f_k(m))\}_{k \in [2^\nu]}$ with a failure probability no more than $(K + 1) \cdot p_{\text{fail}}$ under the following conditions.*

1. $\forall i \in [K + 1], \delta_i \geq \left\lceil \text{erfc}^{-1}(p_{\text{fail}}) \cdot \sqrt{2 \cdot (n/24 + 1/12) + 2(\frac{\alpha_i t}{q})^2 \cdot V_{\text{in}, \text{cmeta}}} \right\rceil$.
2. $\forall i \in [K], (2T_i + 1 + (2^\nu - 1)D_i + \epsilon_i) \cdot \beta_i \leq N, T_i \geq \delta_i + \lfloor \frac{r_i}{2} \rfloor$.
3. $r_0 \leq \frac{2N \cdot 2^{-\nu}}{t}$ and $\forall i \in [K], r_{i+1} \leq r_i \cdot \beta_i$.
4. $r_K \geq 2\delta_K + 1$.

Theorem 5 presents the conditions for correctness of Algorithm 2, whose proof is similar to that of Meta-PBS and is given in Supplementary Materials A. The output noise variance of Meta-PBSManyLUTs is also identical to that of Meta-PBS.

5.2 Meta-WoPPBSManyLUTs

To realize a non-negacyclic function $f : \mathbb{Z}_t \rightarrow \mathbb{Z}_t$, we first extend it to a negacyclic function $f^* : \mathbb{Z}_{2t} \rightarrow \mathbb{Z}_t$ that agrees with f on $[t]$. Then, we use Meta-PBSManyLUTs to obtain encryptions of $\frac{q}{t}f^*(m + \gamma t)$ and $\frac{q}{2}\gamma$, with γ as the most significant bit of $[\psi^*(c)]_{2q}$. Finally, we use a few TruncRepeats and one blind rotation to cancel the effect of γ on f^* , resulting in an encryption of $\frac{q}{t}f^*(m + \gamma t) \cdot (-1)^\gamma = \frac{q}{t}f(m)$. This way of removing γ is similar to FDFB-CancelSign in [24]. Meta-WoPPBS can be turned into Meta-WoPPBSManyLUTs by using $2^\nu > 2$ in the Meta-PBSManyLUTs stage. The full algorithm is presented in Algorithm 3.

Algorithm 3 Meta-WoPPBSManyLUTs

Require: Meta-PBS parameters from Table 1
Require: Parameters ν and D for Meta-PBSManyLUTs
Require: Parameter τ , the number of messages processed in a single CancelSign
Require: Parameter δ_{CS} , β_{CS} and ϵ_{CS} used in TruncRepeat* before CancelSign
Require: $2^\nu - 1$ arbitrary functions $\{f_k\}_{k \in [2^\nu - 1]}$ over $\mathbb{Z}_t$
Require: Input LWE ciphertext $c = \text{LWE}_s(\frac{q}{t}m; e)$ with $\text{Var}(e) \leq V_{\text{in}, \text{Cmeta}}$
Require: Blind rotation keys $\text{BRK}^{(B_{\text{br}}, l_{\text{br}})}$
Ensure: $c_{\text{out}, k} = \text{LWE}_s(\frac{q}{t}f_k(m))$ for $k \in [2^\nu - 1]$
1: Define $f_{2^\nu - 1} : \mathbb{Z}_t \rightarrow \mathbb{Z}_t$ as the constant function $f_{2^\nu - 1}(x) = t/2$
2: $\text{TV} \leftarrow \sum_{i \in [t]} \sum_{k \in [2^\nu]} X^{i \cdot N \cdot 2^{-\nu} / t + k \cdot r_0} \frac{q}{t} f_k(i) \cdot \sum_{j \in [r_0]_{\text{sym}}} X^j$
3: $c_{\text{quo}, 0}, c_{\text{rem}, 0} \leftarrow \text{HomDivRem}(c, N)$
4: $C_0 \leftarrow \text{BlindRotate}(c_{\text{quo}, 0}, \text{TV}, \text{BRK}^{(B_{\text{br}}, l_{\text{br}})})$
5: **for** $i \in [K]$ **do**
6: $C'_i \leftarrow \text{TruncRepeat}^*(C_i, \llbracket -T_i, T_i + D_i \cdot (2^\nu - 1) \rrbracket, \beta_i, \epsilon_i) \cdot X^{\Delta_{r_i \beta_i}}$
7: $c_{\text{quo}, i+1}, c_{\text{rem}, i+1} \leftarrow \text{HomDivRem}(c_{\text{rem}, i}, \beta_i)$
8: $C_{i+1} \leftarrow \text{BlindRotate}(c_{\text{quo}, i+1}, C'_i, \text{BRK}^{(B_{\text{br}}, l_{\text{br}})})$
9: **end for**
10: $c_{\text{sgn}} \leftarrow \text{ModSwitch}(\text{KeySwitch}(\text{SampleExtract}(C_K, D_K(2^\nu - 1)), \text{KSK}_{S \rightarrow s}), 2N)$
11: **for** $j \in [\llbracket \frac{2^\nu - 1}{\tau} \rrbracket]$ **do**
12: $C^* = \sum_{k \in \llbracket j\tau, \min((j+1)\tau, 2^\nu - 2) \rrbracket} \text{TruncRepeat}^*(C_k \cdot X^{-k \cdot D_K}, [r_K - 2\delta_K]_{\text{sym}}, \beta_{CS}, \epsilon_{CS}) \cdot X^{\Delta_{(r_K - 2\delta_K, \beta_{CS})}} \cdot X^{(k - j\tau) \cdot \lfloor N/\tau \rfloor}$
13: $C_{\text{out}} \leftarrow \text{BlindRotate}(c_{\text{sgn}}, C^*, \text{BRK}^{(B_{\text{br}}, l_{\text{br}})})$
14: **for** $k \in \llbracket j\tau, \min((j+1)\tau, 2^\nu - 2) \rrbracket$ **do**
15: $c_{\text{out}, k} \leftarrow \text{SampleExtract}(C_{\text{out}}, (k - j\tau) \cdot \lfloor N/\tau \rfloor)$
16: **end for**
17: **end for**

Corollary 3. *With all the conditions in Theorem 5 (except for using $r_0 \leq \frac{N \cdot 2^{-\nu}}{t}$ and $\alpha_i = \frac{N}{t} \prod_{j \in [i]} \beta_j$), if we additionally have*

$$\delta_{CS} \geq \text{erfc}^{-1}(p_{\text{fail}}) \cdot \sqrt{2} \cdot \sqrt{\left(\frac{2N}{q}\right)^2 (V_{\text{Meta-PBS}} + V_{\text{ks}}) + V_{\text{ms}}},$$

$$2\delta_{\text{CS}} + 1 \leq (r_K - 2\delta_K)\beta_{\text{CS}},$$

$$(r_K - 2\delta_K + \epsilon_{\text{CS}})\beta_{\text{CS}} \leq \lfloor \frac{N}{\tau} \rfloor.$$

then Algorithm 3 outputs $\{\text{LWE}_{\mathbf{s}}(\frac{q}{t}f_k(m))\}_{k \in [2^\nu - 1]}$ with a failure probability of $(K + 2) \cdot p_{\text{fail}}$.

Proof. Line 1 to 9 evaluates a Meta-PBSManyLUTs (without the SampleExtraction part) with 2^ν negacyclic functions $f_k^* : \mathbb{Z}_{2t} \rightarrow \mathbb{Z}_t$ on $[c]_{2q} = \text{LWE}_{\mathbf{s}}(\frac{q}{t}m + q\gamma; e)$. For $x \in [t]$, $f_k^*(x) = f_k(x) \in \mathbb{Z}_t$ with $k \in [2^\nu - 1]$, and $f_{2^\nu - 1}^*(x) = t/2$. Thus, $\text{Dec}(C_K) \cdot X^{-k \cdot D_K}$ is $(r_K - 2\delta_K)$ -redundant in $f_k^*(m + \gamma t) = f_k(m) \cdot (-1)^\gamma$.

$c_{\text{sgn}} = \text{LWE}_{\mathbf{s}}(N\gamma; e_{\text{sgn}})$ with $\text{Var}(e_{\text{sgn}}) = (\frac{2N}{q})^2(V_{\text{Meta-PBS}} + V_{\text{ks}}) + V_{\text{ms}}$. Thus, $\Pr[|e_{\text{sgn}}| > \delta_{\text{CS}}] < p_{\text{fail}}$.

In Line 12 to 13, the third condition ensures the supports of different TruncRepeats do not overlap. $\text{Dec}(C^*) \cdot X^{-(k-j\tau) \cdot \lfloor N/\tau \rfloor}$ is then $(r_K - 2\delta_K)\beta_{\text{CS}}$ -redundant in $\frac{q}{t}f_k(m) \cdot (-1)^\gamma$. Since $\text{Dec}(C_{\text{out}}) = \text{Dec}(C^*) \cdot X^{-N\gamma - e_{\text{sgn}}}$, $\text{Dec}(C_{\text{out}}) \cdot X^{-(k-j\tau) \cdot \lfloor N/\tau \rfloor}$ is $((r_K - 2\delta_K)\beta_{\text{CS}} - 2\delta_{\text{CS}})$ -redundant in $\frac{q}{t}f_k(m)$.

Finally, the correctness of SampleExtract is guaranteed by $(r_K - 2\delta_K)\beta_{\text{CS}} - 2\delta_{\text{CS}} \geq 1$. $\square$

Corollary 4. *The output ciphertexts of Algorithm 3 have a noise variance of*

$$V_{\text{Meta-WoPPBSManyLUTs}} = V_{\text{Meta-PBS}} + V_{\text{br}} + V_{\text{tr}, \epsilon_{\text{CS}}} + (\tau - 1)V_{\text{tr}, \text{empty}, \epsilon_{\text{CS}}}.$$

Proof. The useful coefficients in C_γ have the following noise components.

- The noise from C_K , with a variance of $V_{\text{Meta-PBS}}$.
- The noise from TruncRepeat* that set this coefficient, with a variance of $V_{\text{tr}, \epsilon_{\text{CS}}}$.
- The noise from other $\tau - 1$ TruncRepeat*, with a total variance of $(\tau - 1)V_{\text{tr}, \text{empty}, \epsilon_{\text{CS}}}$.

The final blind rotation adds a variance of V_{br} . $\square$

5.3 Put It Together: Supporting Large-Scale Homomorphic Linear Combinations after Meta-WoPPBS

If we want to evaluate a length- L integer combination on outputs of many Meta-WoPPBSManyLUTs before feeding the result into the next Meta-WoPPBS,

$$L \frac{t^2}{4} \cdot V_{\text{Meta-WoPPBSManyLUTs}} + V_{\text{ks}} \leq V_{\text{in}, c_{\text{meta}}},$$

where the amplification factors $\frac{t^2}{4}$ and L come from scalar multiplication in $\mathbb{Z}_t$ and the L additions, respectively. As indicated by the equation, the rapid noise growth due to multiplication by constants in $\mathbb{Z}_t$ requires a small bootstrapping noise, leading to less efficient bootstrapping parameters.

To overcome this issue, we output a Lev ciphertext of $f(m)$ with gadget length $l_{\text{msg}} = 2^\nu - 1$ using Meta-WoPPBSManyLUTs to reduce the noise growth. Multiplication by $v \in \mathbb{Z}_t$ is then realized by

$$\left\langle G^{(B_{\text{msg}}, l_{\text{msg}}, t)}(c), \text{Meta-WoPPBSManyLUTs} \left(\text{LWE} \left(\frac{q}{t} m \right), g^{(B_{\text{msg}}, l_{\text{msg}}, t)} \cdot f(\cdot) \right) \right\rangle^4.$$

Note that the gadget decomposition here needs to be an exact one for correctness. Now, the noise growth of length- L linear combinations turns into

$$L \cdot \max_{v \in \mathbb{Z}_t} \left(\left\| G^{(B_{\text{msg}}, l_{\text{msg}}, t)}(v) \right\|_2^2 \right) \cdot V_{\text{Meta-WoPPBSManyLUTs}} + V_{\text{ks}} \leq V_{\text{in}, c_{\text{meta}}}.$$

For the example of $t = 2^8$ and $\nu = 2$, we could use decomposition bases of $2^3, 2^3$ and 2^2 , thus reducing the amplification factor of $V_{\text{Meta-WoPPBSManyLUTs}}$ from 2^{14} to $\sqrt{4^2 + 4^2 + 2^2} = 6$.

This can be generalized to linear combinations with multiple layers by outputting Lev ciphertexts after each linear combination, but the input Lev ciphertexts will have correlated LWE noises starting from the second layer. Thus, we need to record the linear-combination coefficients of each LWE ciphertext with respect to the freshly extracted ciphertexts to compute the noise variance. Note that the gadget parameters of the non-freshly-extracted Lev ciphertexts can be freely chosen regardless of B_{msg} and l_{msg} . Taking a decomposition base of two, each layer (starting from the second one) amplifies the noise variance by a factor of at most $(L \cdot \log_2 t)^2$.

However, outputting a Lev ciphertext instead of a single LWE ciphertext has several drawbacks. Most importantly, since the number of distinct LUT entries in the initial test vector increases from $2t$ to $l_{\text{msg}}t$, the maximum value of t decreases from $\frac{N}{2}$ to $\frac{N}{l_{\text{msg}}}$, making this approach less suitable for high-precision inputs. Secondly, the computation cost and memory usage increase because Meta-PBS may require more blind rotations to compensate for the smaller initial redundancy, and also because the linear combination between Lev ciphertexts is about l_{msg} times more expensive than that between LWE ciphertexts.

For the completeness of definition, we define the homomorphic capacity for this scenario as

$$c_{\text{Hom}} = \log_2 \left(\sqrt{\frac{V_{\text{in}, c_{\text{meta}}} - V_{\text{ks}}}{V_{\text{Meta-WoPPBSManyLUTs}}}} \right).$$

6 Implementation

6.1 Automatic Parameter Selection

Meta-PBS has many parameters that have a complicated impact on the overall performance. For example, choosing the largest β_i greedily in order may seem

⁴ When $B_{\text{msg}} \nmid \log_2(t)$, we could use different decomposition bases for each digit, instead of using the same B_{msg} to obtain lower noise growth

to be a good option to obtain the lowest K , thus decreasing the number of required blind rotations. However, large β_i 's in the early iterations will lead to the fast growth of r_i , δ_i , and T_i , which will force us to use small β_i 's in later iterations, resulting in non-optimal K . The final redundancy r_K in Meta-WoPPBSManyLUTs provides another performance tradeoff. A larger r_K helps to use larger ϵ_i 's in the following TruncRepeat*, but could potentially increase the cost of the Meta-PBSManyLUTs.

Considering the difficulty in selecting the optimal parameters, we propose a heuristic parameter selection algorithm to automate this process. Due to limited room, the details are given in Supplementary Materials C.

6.2 Comparing with Other LUT Evaluation Approaches

We compare Meta-PBS with EBS, single-ciphertext integer-input LHE, and the tree-based method in their efficiency in evaluating large LUTs and their capability to perform homomorphic arithmetic (i.e., $\mathbb{Z}$ -linear combinations) after bootstrapping. All experiments are conducted on a workstation with Intel(R) Xeon(R) Gold 6248R CPU @ 3.00GHz and Fedora release 38. Our implementation is developed on TFHE-go⁵ [19]. All LWE/RLWE parameters (including those in existing methods) are decided or updated to meet 130 bits of security using the Lattice Estimator⁶ [1] with `red_cost_model=BDGL16`, in alignment with TFHE-rs [31]. The timing results are averaged over 1000 runs unless otherwise specified.

Comparing with EBS [21]. Currently, the EBS implementation only supports the evaluation of negacyclic LUTs. Thus, the experiment is limited to evaluating negacyclic functions, with the parameters decided as follows.

- For $\log_2(t) \in \llbracket 8, 12 \rrbracket$, find parameters for EBS that can evaluate t -bit negacyclic LUT with $c_{\text{Hom}} \geq 0.5$. This setting of c_{Hom} only supports a single ciphertext addition after bootstrap, which is the minimum requirement for the scheme to be fully homomorphic.
- For $\log_2(t) \in \llbracket 8, 12 \rrbracket$, find parameters for Meta-PBS with a c_{Hom} no smaller than that of EBS with the same t .

Following the default settings in TFHE-go, we set the failure probability of EBS and Meta-PBS to 2^{-64} . Parameters for both schemes are obtained using Algorithm 4 and are given in Table 2 and Table 3.

The benchmark results are given in Table 4. As expected, the running time of EBS increases linearly with t , while the running time of Meta-PBS grows more smoothly, leading to a speedup of 3.1-79.6x. However, due to the existence of TruncRepeat keys, Meta-PBS has a larger evaluation key size, especially when ϵ_i 's are small. Note that Meta-PBS runs slower with $t = 2^{11}$ than with $t = 2^{12}$ because the former needs to ensure a higher c_{Hom} to match the c_{Hom} of EBS.

⁵ <https://github.com/sp301415/tfhe-go/tree/2369903>

⁶ <https://github.com/malb/lattice-estimator/tree/5ba00f5>

Table 2: LWE/RLWE parameters and p_{fail} for EBS and Meta-PBS. Note that we use a smaller p_{fail} for Meta-PBS so that the failure probability of a whole Meta-PBS matches that of EBS.

n	N	q	σ_{LWE}/q	σ_{RLWE}/q	p_{fail}
1170	2048	2^{64}	$2^{-27.44}$	$2^{-50.22}$	2^{-64} for EBS and 2^{-66} for Meta-PBS

Table 3: Detailed parameters for EBS and Meta-PBS with $\log_2(t) \in [8, 12]$. The first (second) row for each t corresponds to EBS (Meta-PBS). Blind rotation in EBS is performed in a ring dimension of kN , while Meta-PBS always uses $k = 1$. A single B_{tr} value in the cell means the value is shared across all TruncRepeat*.

t	k	B_{br}	l_{br}	B_{ks}	l_{ks}	c_{Hom}	K	β	r	δ	T	ϵ	c_{meta}	B_{tr}	l_{tr}
2^8	8	2^{15}	2	2^5	5	3.92	/	/	/	/	/	/	/	/	/
	1	2^{15}	2	2^7	3	4.67	1	(9)	(15,135)	(67,67)	(73)	(80)	2.16	2^{23}	1
2^9	16	2^{15}	2	2^4	6	2.81	/	/	/	/	/	/	/	/	/
	1	2^{15}	2	2^9	2	4.80	2	(5,4)	(8,37,148)	(66,66,73)	(70,84)	(268,343)	1.27	2^{15}	2
2^{10}	32	2^{15}	2	2^3	8	1.46	/	/	/	/	/	/	/	/	/
	1	2^{15}	2	2^7	3	5.12	2	(6,6)	(4,24,144)	(66,66,71)	(68,78)	(204,184)	1.34	2^{15}	2
2^{11}	128	2^{15}	2	2^7	3	6.11	/	/	/	/	/	/	/	/	/
	1	2^{11}	3	2^6	4	7.32	2	(9,8)	(2,18,144)	(66,66,71)	(67,75)	(92,105)	1.34	2^{12}	3
2^{12}	256	2^{15}	2	2^7	3	3.77	/	/	/	/	/	/	/	/	/
	1	2^{15}	2	2^5	5	3.83	2	(14,12)	(1,14,168)	(66,66,83)	(66,73)	(13,23)	0.71	2^{11}	3

Comparing with Single-ciphertext Integer-input LHE [29]. Integer-Input LHE breaks the input ciphertext with plaintext modulus t into $\log_2(t)$ ciphertexts that encrypt the bits of the plaintext. Then, CBS is used to convert these LWE ciphertexts into RGSW ciphertexts. An arbitrary LUT modulo t could be evaluated through $\log_2(t)$ external products on a test vector. We compare Meta-WoPPBSManyLUTs with integer-input LHE using $t = 2^8$. Although integer-input LHE could work with digit-decomposed inputs, we require that it uses a single 8-bit input ciphertext for the fairness of comparison. Specifically, the parameter set with $\delta = 8$ and $\tau = 2$ is adopted (see Table 7 of [29]), where the input ciphertext is digit-decomposed under basis 2^δ and each bootstrapping extracts and refreshes τ plaintext bits.

We did not implement integer-input LHE in TFHE-go, but used the existing implementation based on TFHE-rs⁷ instead. A potential problem is that the running time of the same operation may vary in different libraries, thus affecting the fairness of comparison. We address this issue by benchmarking the Extr.+Refr. part of integer-input LHE (i.e., a blind rotation followed by a key switching)

⁷ <https://github.com/KAIST-CryptLab/refined-tfhe-lhe/tree/9b0426e>

Table 4: The running time, homomorphic capacity, and evaluation key size for EBS and Meta-PBS. Following [29], the evaluation keys are assumed to be compressed. I.e., all the $\mathbf{a}$ in LWE ciphertexts and A parts in RLWE ciphertexts are replaced by a single random seed. Timing for EBS with $t = 2^{11}$ and $t = 2^{12}$ are averaged over 100 instead of 1000 runs.

t	PBS Type	Time (ms)	Speedup	c_{Hom}	Evaluation key size (MB)
2^8	EBS	331	1x	3.92	73.16
	Meta-PBS	107	3.1x	4.67	73.54
2^9	EBS	670	1x	2.81	73.17
	Meta-PBS	157	4.3x	4.80	73.51
2^{10}	EBS	1,323	1x	1.46	73.18
	Meta-PBS	157	8.4x	5.12	73.77
2^{11}	EBS	5,368	1x	6.11	73.15
	Meta-PBS	215	24.9x	7.32	111.64
2^{12}	EBS	13,442	1x	3.77	73.15
	Meta-PBS	169	79.6x	3.83	83.99

Table 5: LWE/RLWE parameters and p_{fail} for integer-input LHE and Meta-WoPPBSManyLUTs.

Name	n	N	q	σ_{LWE}/q	σ_{RLWE}/q	p_{fail}
LHE_8	953	2048	2^{64}	$2^{-21.84}$	$2^{-50.22}$	2^{-40}
Meta_8C	970	2048	2^{64}	$2^{-22.28}$	$2^{-50.22}$	2^{-42}
Meta_8						

in TFHE-go, while the running time of the remaining parts is still measured in TFHE-rs.

We configure Meta-WoPPBSManyLUTs to support linear combination after bootstrapping, as discussed in Section 5.3. The maximum supported linear combination size L is $\lfloor 2^{2c_{\text{Hom}}}/36 \rfloor$. For integer-input LHE, we use 8 external products to realize LUT evaluation and constant multiplication (as part of the linear combination) in one step. c_{Hom} for integer-input LHE is defined as the maximum value of c such that a LWE ciphertext with a noise variance of $2^{2c} \times (\text{Error variance after 8 external products})$ can still be bootstrapped correctly. The concrete variance values are obtained using the scripts provided in their implementation. Note $L = \lfloor 2^{2c_{\text{Hom}}} \rfloor$ in integer-input LHE.

The parameter sets are given in Table 5 and Table 6. LHE_8 is the parameter set from integer-input LHE. Meta_8C and Meta_8 are based on our methods, where the latter trades computation time for a larger L . The overall failure probability is set to 2^{-40} following the settings in Refined LHE. The

Table 6: Detailed parameters for integer-input LHE and Meta-WoPPBSManyLUTs with $t = 2^8$. Some parameters exclusive to integer-input LHE are not displayed, e.g., gadget parameters for HomTrace and RGSW ciphertexts.

Name	B_{br}	l_{br}	B_{ks}	l_{ks}	c_{Hom}	K	β, β_{CS}	r, r_{ext}	$\delta, \delta_{\text{CS}}$	T	$\epsilon, \epsilon_{\text{CS}}$	c_{meta}	B_{tr}	l_{tr}	ν	τ
LHE_8	2^9	4	2^7	2	4.91	/	/	/	/	/	/	/	/	/	/	/
Meta_8C	2^{15}	2	2^2	10	6.68	2	$\begin{smallmatrix} (17,8) \\ 16 \end{smallmatrix}$	$\begin{smallmatrix} (2,34,272) \\ 6 \end{smallmatrix}$	$\begin{smallmatrix} (47,50,133) \\ 47 \end{smallmatrix}$	$\begin{smallmatrix} (48,67) \\ (17,19) \\ 35 \end{smallmatrix}$	$\begin{smallmatrix} (17,19) \\ 35 \end{smallmatrix}$	2.13	2^{11}	3	2	3
Meta_8	2^{11}	3	2^2	10	9.90	2	$\begin{smallmatrix} (15,8) \\ 16 \end{smallmatrix}$	$\begin{smallmatrix} (2,30,240) \\ 6 \end{smallmatrix}$	$\begin{smallmatrix} (47,49,116) \\ 47 \end{smallmatrix}$	$\begin{smallmatrix} (48,64) \\ (33,37) \\ 35 \end{smallmatrix}$	$\begin{smallmatrix} (33,37) \\ 35 \end{smallmatrix}$	2.17	2^{11}	3	2	3

benchmark results are given in Table 7. For the Extr.+Refr. part in LHE_8, its implementation in TFHE-go and TFHE-rs takes 74.4 ms and 64.7 ms, respectively. Recall that the timing of this part is measured in TFHE-go to ensure a fair comparison.

Table 7: The running time, maximum linear combination size L , and (compressed) evaluation key size for integer-input LHE and Meta-WoPPBSManyLUTs. Time for performing linear combinations is not included here.

Name	Time (ms)	Speedup	L	Evaluation key size (MB)
LHE_8	543	1x	905	120.45
Meta_8C	208	2.61x	>290	73.41
Meta_8	266	2.05x	>25000	98.94

As shown in the table, our method demonstrates a 2.06-2.63x speedup compared to LHE_8. This speedup comes from two reasons. First, the additive noise growth of our method enables us to set l_{br} to 2 or 3, while LHE_8 has to use $l_{\text{br}} = 4$ for noise control because CBS introduces a multiplicative noise variance growth of factor $O(N)$. As both methods require 4 blind rotations, the smaller l_{br} in our method reduces the total time for blind rotations. Second, the running time of our method is dominated by blind rotations, with other operations taking about 30 ms in total. In contrast, integer-input LHE needs to perform CBS after Extr.+Refr., which is an expensive operation.

Apart from advantages in running time, the additive noise growth in our method enables Meta_8 to support a much larger L than LHE_8. Our method also has a slightly smaller evaluation key size compared to integer-input LHE.

Remark. Integer-input LHE could work with digit-decomposed input/output ciphertexts to obtain better performance, e.g., using two input ciphertexts with

$t = 2^4$. However, digit-decomposed inputs may not always be available, depending on the upstream homomorphic computation. If this is the case, expensive homomorphic digit decomposition is required to convert the inputs into the decomposed format [22]. Thus, our method will be a better choice when a non-decomposed plaintext representation is adopted. However, LHE/CBS is more favorable for binary arithmetic or multivariate LUT evaluation.

Table 8: The running time, maximum linear combination size L , and (compressed) evaluation key size for the tree-based method and Meta-WoPPBSManyLUTs.

Name	Time (ms)	Speedup	L	Evaluation key size (MB)
Tree-based method	422	1x	NA	160
Meta_8C	208	2.03X	>290	73.41
Meta_8	266	1.59x	>25000	98.29

Comparing with the Tree-based Method [16,28]. We compare Meta-WoPPBSManyLUTs with the tree-based 8-bit arbitrary LUT evaluation from [28], which uses a plaintext decomposition basis of $B_{\text{tree}} = 2^4$. Since the implementation of [28] is not publicly available, we use the same method in [29] to estimate its running time. That is, we benchmark blind rotation and public functional key switching in TFHE-go, and calculate the running time of 8-bit LUT evaluation by counting the number of blind rotations and public functional key switchings (which are three and two here, respectively).

The benchmark results obtained are presented in Table 8⁸, which shows that our method achieves a speedup of 1.59-2.03x. Although the tree-based method requires only three blind rotations, it needs to use $l_{\text{br}} = 3$ to accommodate the $O(t^3)$ noise growth in multi-value bootstrapping [6] (or $O(t^2)$ using the multi-value extraction technique [16]). Its public functional key-switchings are also more expensive than our optimized TruncRepeat*. Besides disadvantages in running time, the tree-based method is unable to support homomorphic addition or multiplication with constants due to its digit-decomposed representation of plaintexts, making it less suitable for large-scale homomorphic linear combinations. However, when it comes to multivariate LUT evaluation, the use of decomposed inputs makes the tree-based method a potentially better choice than Meta-PBS.

6.3 Validating the Noise Analysis

Although the blind rotation error has been verified to be a centered Gaussian [9], the design of Meta-PBS brings several subtleties to its noise analysis. Specifically,

⁸ Note that the tree-based method uses $p_{\text{fail}} = 2^{-23}$, while Meta_8 and Meta_8C still guarantee a failure probability of 2^{-40} .

multiple blind rotations are performed on the same RLWE ciphertext with the same blind rotation keys, and the coefficients of noise terms are duplicated by TruncRepeat, which may introduce dependency between different contributing noise components and/or coefficients of the output noise.

This raises two questions: (1) if each coefficient in the output noise is a centered Gaussian with the expected standard deviation, and (2) if the coefficients in the output noise are independent.

For the first question, we find that the output noise is a biased Gaussian with below-estimation variance, where the bias and variance have library-dependent values. We propose a heuristic and efficient fix to remove the bias in the output noise by randomizing the CMux gates in blind rotation. For the second question, we argue that the independence between noise coefficients follows from commonly used independence assumptions. More details are provided in Supplementary Materials D.

References

1. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology* **9**(3), 169–203 (2015). <https://doi.org/doi:10.1515/jmc-2015-0016>
2. Bae, Y., Cheon, J.H., Cho, W., Kim, J., Kim, T.: META-BTS: Bootstrapping precision beyond the limit. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) *ACM CCS 2022: 29th Conference on Computer and Communications Security*. pp. 223–234. ACM Press, Los Angeles, CA, USA (Nov 7–11, 2022). <https://doi.org/10.1145/3548606.3560696>
3. Bergerat, L., Boudi, A., Bourgerie, Q., Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Parameter optimization and larger precision for (T)FHE. *Journal of Cryptology* **36**(3), 28 (Jul 2023). <https://doi.org/10.1007/s00145-023-09463-5>
4. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) *Advances in Cryptology – CRYPTO 2012. Lecture Notes in Computer Science*, vol. 7417, pp. 868–886. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 19–23, 2012). https://doi.org/10.1007/978-3-642-32009-5_50
5. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. *ACM Trans. Comput. Theory* **6**(3) (Jul 2014). <https://doi.org/10.1145/2633600>
6. Carpov, S., Izabachène, M., Mollimard, V.: New techniques for multi-value input homomorphic evaluation and applications. In: Matsui, M. (ed.) *Topics in Cryptology – CT-RSA 2019. Lecture Notes in Computer Science*, vol. 11405, pp. 106–126. Springer, Cham, Switzerland, San Francisco, CA, USA (Mar 4–8, 2019). https://doi.org/10.1007/978-3-030-12612-4_6
7. Chen, H., Dai, W., Kim, M., Song, Y.: Efficient homomorphic conversion between (ring) LWE ciphertexts. In: Sako, K., Tappenhauer, N.O. (eds.) *ACNS 2021: 19th International Conference on Applied Cryptography and Network Security, Part I. Lecture Notes in Computer Science*, vol. 12726, pp. 460–479. Springer, Cham, Switzerland, Kamakura, Japan (Jun 21–24, 2021). https://doi.org/10.1007/978-3-030-78372-3_18

8. Cheon, J.H., Cho, W., Kim, J., Stehlé, D.: Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) *ACM CCS 2023: 30th Conference on Computer and Communications Security*. pp. 696–710. ACM Press, Copenhagen, Denmark (Nov 26–30, 2023). <https://doi.org/10.1145/3576915.3623086>
9. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology* **33**(1), 34–91 (Jan 2020). <https://doi.org/10.1007/s00145-019-09319-x>
10. Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2021, Part III. Lecture Notes in Computer Science*, vol. 13092, pp. 670–699. Springer, Cham, Switzerland, Singapore (Dec 6–10, 2021). https://doi.org/10.1007/978-3-030-92078-4_23
11. Ducas, L., Micciancio, D.: FHEW: Bootstrapping homomorphic encryption in less than a second. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology – EUROCRYPT 2015, Part I. Lecture Notes in Computer Science*, vol. 9056, pp. 617–640. Springer Berlin Heidelberg, Germany, Sofia, Bulgaria (Apr 26–30, 2015). https://doi.org/10.1007/978-3-662-46800-5_24
12. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, Report 2012/144 (2012), <https://eprint.iacr.org/2012/144>
13. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Mitzenmacher, M. (ed.) *41st Annual ACM Symposium on Theory of Computing*. pp. 169–178. ACM Press, Bethesda, MD, USA (May 31 – Jun 2, 2009). <https://doi.org/10.1145/1536414.1536440>
14. Gentry, C., Halevi, S., Peikert, C., Smart, N.P.: Ring switching in BGV-style homomorphic encryption. In: Visconti, I., De Prisco, R. (eds.) *SCN 12: 8th International Conference on Security in Communication Networks. Lecture Notes in Computer Science*, vol. 7485, pp. 19–37. Springer Berlin Heidelberg, Germany, Amalfi, Italy (Sep 5–7, 2012). https://doi.org/10.1007/978-3-642-32928-9_2
15. Gentry, C., Halevi, S., Smart, N.P.: Better bootstrapping in fully homomorphic encryption. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) *PKC 2012: 15th International Conference on Theory and Practice of Public Key Cryptography. Lecture Notes in Computer Science*, vol. 7293, pp. 1–16. Springer Berlin Heidelberg, Germany, Darmstadt, Germany (May 21–23, 2012). https://doi.org/10.1007/978-3-642-30057-8_1
16. Guimarães, A., Borin, E., Aranha, D.F.: Revisiting the functional bootstrap in TFHE. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2021**(2), 229–253 (2021). <https://doi.org/10.46586/tches.v2021.i2.229-253>, <https://tches.iacr.org/index.php/TCHES/article/view/8793>
17. Halevi, S., Shoup, V.: Algorithms in HELib. In: Garay, J.A., Gennaro, R. (eds.) *Advances in Cryptology – CRYPTO 2014, Part I. Lecture Notes in Computer Science*, vol. 8616, pp. 554–571. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 17–21, 2014). https://doi.org/10.1007/978-3-662-44371-2_31
18. Halevi, S., Shoup, V.: Bootstrapping for HELib. *Journal of Cryptology* **34**(1), 7 (Jan 2021). <https://doi.org/10.1007/s00145-020-09368-7>
19. Hwang, I.: TFHE-go. Online: <https://github.com/sp301415/tfhe-go> (2023)
20. Lee, K.H., Yoon, J.W.: Homomorphic Field Trace Revisited : Breaking the Cubic Noise Barrier. *Cryptology ePrint Archive*, Paper 2025/1088 (2025), <https://eprint.iacr.org/2025/1088>, to appear at TCHES 2026.

21. Lee, K., Yoon, J.W.: Discretization error reduction for high precision torus fully homomorphic encryption. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science, vol. 13941, pp. 33–62. Springer, Cham, Switzerland, Atlanta, GA, USA (May 7–10, 2023). https://doi.org/10.1007/978-3-031-31371-4_2
22. Liu, Z., Micciancio, D., Polyakov, Y.: Large-precision homomorphic sign evaluation using FHEW/TFHE bootstrapping. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology – ASIACRYPT 2022, Part II. Lecture Notes in Computer Science, vol. 13792, pp. 130–160. Springer, Cham, Switzerland, Taipei, Taiwan (Dec 5–9, 2022). https://doi.org/10.1007/978-3-031-22966-4_5
23. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Gilbert, H. (ed.) Advances in Cryptology – EUROCRYPT 2010. Lecture Notes in Computer Science, vol. 6110, pp. 1–23. Springer Berlin Heidelberg, Germany, French Riviera (May 30 – Jun 3, 2010). https://doi.org/10.1007/978-3-642-13190-5_1
24. Ma, S., Huang, T., Wang, A., Zhou, Q., Wang, X.: Fast and accurate: Efficient full-domain functional bootstrap and digit decomposition for homomorphic computation. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(1), 592–616 (2024). <https://doi.org/10.46586/tches.v2024.i1.592-616>
25. Nishimura, T., Tozawa, K., Sadakane, K.: Surpassing the Word Size Limitation of TFHE with Noise Calibration. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. p. 438–452. CCS ’25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3719027.3744808>, <https://doi.org/10.1145/3719027.3744808>
26. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. J. ACM **56**(6) (Sep 2009). <https://doi.org/10.1145/1568318.1568324>
27. Rivest, R.L., Adleman, L., Dertouzos, M.L., et al.: On data banks and privacy homomorphisms. Foundations of secure computation **4**(11), 169–180 (1978)
28. Trama, D., Boudguiga, A., Clet, P.E., Sirdey, R., Ye, N.: Designing a general-purpose 8-bit (t)fhe processor abstraction. IACR Transactions on Cryptographic Hardware and Embedded Systems **2025**(2), 535–578 (Mar 2025). <https://doi.org/10.46586/tches.v2025.i2.535-578>
29. Wang, R., Ha, J., Shen, X., Lu, X., Chen, C., Wang, K., Lee, J.: Refined TFHE Leveled Homomorphic Evaluation and Its Application. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. p. 2309–2323. CCS ’25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3719027.3744873>, <https://doi.org/10.1145/3719027.3744873>
30. Wang, R., Wen, Y., Li, Z., Lu, X., Wei, B., Liu, K., Wang, K.: Circuit bootstrapping: Faster and smaller. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024, Part II. Lecture Notes in Computer Science, vol. 14652, pp. 342–372. Springer, Cham, Switzerland, Zurich, Switzerland (May 26–30, 2024). https://doi.org/10.1007/978-3-031-58723-8_12
31. Zama: TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data (2022), <https://github.com/zama-ai/tfhe-rs>

Supplementary Materials

A Proofs

Proof (Lemma 3).

– First, $\forall x \in \mathbb{Z}$,

$$\begin{aligned} & \text{Trunc}(m, \{a\}) \cdot X^i \\ &= m[[a]_N] \cdot X^{[a]_N + i} \\ &= \text{Trunc}(m[[a]_N] \cdot X^{[a]_N + i}, \{a + i\}) \\ &= \text{Trunc}(m \cdot X^i, \{a + i\}). \end{aligned}$$

Using Lemma 2,

$$\text{Trunc}(m, S) \cdot X^i = \sum_{a \in S} \text{Trunc}(m, \{a\}) \cdot X^i = \text{Trunc}(m \cdot X^i, S + i).$$

- The conclusion holds because $\mathbb{Z}_q[X, X^{-1}]/(X^N + 1) \cong R_q$ and Embed is a ring homomorphism.
- Embed sends both sides to $m \cdot X^i$, so $(X^N + 1) \mid (\text{Lift}(m \cdot X^i) - \text{Lift}(m) \cdot X^i)$. The diameter of $\text{supp}(\text{Lift}(m \cdot X^i) - \text{Lift}(m) \cdot X^i)$ is less than N since

$$\begin{aligned} & \text{supp}(\text{Lift}(m \cdot X^i) - \text{Lift}(m) \cdot X^i) \\ & \subseteq \text{supp}(\text{Lift}(m \cdot X^i)) \cup \text{supp}(\text{Lift}(m) \cdot X^i) \subseteq [N]_{\text{sym}}. \end{aligned}$$

The conclusion follows because $\text{supp}(X^N + 1)$ has a diameter of N . □

Proof (Lemma 4).

$$\begin{aligned} & \text{Embed}(\pi_B(\text{Lift}(\text{Trunc}(M \cdot X^i, \llbracket a, b \rrbracket)))) \\ &= \text{Embed}(\pi_B(\text{Lift}(\text{Trunc}(M, \llbracket a - i, b - i \rrbracket) \cdot X^i))) \\ &= \text{Embed}(\pi_B(\text{Lift}(\text{Trunc}(M, \llbracket a - i, b - i \rrbracket)) \cdot X^i)) \\ &= \text{Embed}(\pi_B(\text{Lift}(\text{Trunc}(M, \llbracket a - i, b - i \rrbracket)))) \cdot X^{iB}, \end{aligned}$$

where all the equality follow from Lemma 3. The second equality also uses

$$\begin{aligned} & \llbracket a - i, b - i \rrbracket_N^{\text{sym}} + i \\ &= [\llbracket a, b \rrbracket_N^{\text{sym}} - i]_N^{\text{sym}} + i \\ &= [\llbracket a, b \rrbracket_N^{\text{sym}} - i + i] \\ &= \llbracket a, b \rrbracket_N^{\text{sym}} \subseteq [N]_{\text{sym}}. \end{aligned} \quad \square$$

Proof (Lemma 5). The case of $B = 1$ is straightforward. For $B \geq 2$,

$$\begin{aligned}
& \text{Embed}(\pi_B(\text{Lift}(\text{Trunc}(m, \llbracket a, b \rrbracket)))) \\
&= \text{Embed}(\pi_B(\text{Lift}(\text{Trunc}(m, \llbracket a, b \rrbracket_{\geq 0})))) + \text{Embed}(\pi_B(\text{Lift}(\text{Trunc}(m, \llbracket a, b \rrbracket_{< 0})))) \\
&= \text{Embed}\left(\sum_{j \in \llbracket a, b \rrbracket_{\geq 0}} m[j] \cdot X^{jB}\right) + \text{Embed}\left(\sum_{j \in \llbracket a, b \rrbracket_{< 0}} -m[N+j] \cdot X^{jB}\right) \\
&= \sum_{j \in \llbracket a, b \rrbracket_{\geq 0}} m[j] \cdot X^{jB} + \sum_{j \in \llbracket a, b \rrbracket_{< 0}} m[N+j] \cdot X^{N+jB}
\end{aligned}$$

The first equality follows from Lemma 2 and π_B being a homomorphism. The second equality also follows from Lemma 2. The third equality is ensured by $|jB| < N$ for $j \in \llbracket a, b \rrbracket$. $\square$

Proof (Lemma 6). According to Lemma 5, for $k \in \llbracket a, b \rrbracket$,

$$\text{supp}(\text{truncRepeat}(M, \{k\}, B) = [kB + [B]_{\text{sym}}]_N.$$

Also, for $k_1 \neq k_2 \in \llbracket a, b \rrbracket$,

$$[k_1B + [B]_{\text{sym}}]_N \cap [k_2B + [B]_{\text{sym}}]_N = \emptyset$$

because $[B]_{\text{sym}}$ has a diameter of $B - 1 \leq N/(b - a + 1) - 1 \leq N/2 - 1$. Thus,

$$\begin{aligned}
& \text{Trunc}(\text{truncRepeat}(M, \llbracket a, b \rrbracket, B) \cdot X^{-jB}, [B]_{\text{sym}}) \\
&= \text{Trunc}\left(\sum_{i \in \llbracket a, b \rrbracket} \text{truncRepeat}(M, \{i\}, B), [B]_{\text{sym}} + jB\right) \cdot X^{-jB} \\
&= \text{Trunc}(\text{truncRepeat}(M, \{j\}, B), [B]_{\text{sym}} + jB) \cdot X^{-jB} \\
&= \text{truncRepeat}(M, \{j\}, B) \cdot X^{-jB} \\
&= \text{truncRepeat}(M \cdot X^{-j}, \{0\}, B) = (M \cdot X^{-j})[0] \cdot \sum_{i \in [B]_{\text{sym}}} X^i. \quad \square
\end{aligned}$$

Proof (Theorem 5). The proof is similar to that of Algorithm 1.

For $i \in [K + 1]$, let $\bar{\beta}_i = \prod_{j \in [i]} \beta_j$ and $\Delta_i = \frac{q}{2N} \cdot \bar{\beta}_i^{-1} = \frac{q}{t} \alpha_i$.

Suppose $C_i = \text{RLWE}_S(M_i)$ for $i \in [K + 1]$, we claim that for all $k \in [2^\nu]$

$$M_i = \bar{M}_i \cdot X^{-e_i}, \text{ where } \bar{M}_i \cdot X^{-kD_i} \text{ is } r_i\text{-redundant in } \frac{q}{t} f_k(m) \text{ and } e_i = \frac{e - \psi^*(c_{\text{rem}, i})}{\Delta_i}.$$

We still have $\Pr[|e_i| > \delta_i] < p_{\text{fail}}$. Let $\bar{M}_0 = \text{TV} \cdot X^{-\frac{2N}{t}m}$ and $e_0 = \psi(c_{\text{quo}, 0}) - \frac{2N}{t}m$. Since $\bar{M}_0 \cdot X^{-k \cdot D_0}$ is r_0 -redundant in $\frac{q}{t} f_k(m)$ by design, the claim holds for $i = 0$.

Assume the claim holds for $i - 1$, then for $i \in [1, K]$,

- Suppose C'_{i-1} encrypts M'_{i-1} , then $M'_{i-1} \cdot X^{-kD_{i-1}\beta_{i-1} + e_{i-1}\beta_{i-1}}$ is $r_{i-1}\beta_{i-1}$ -redundant in $\frac{q}{t} f_k(m)$ using $|e_{i-1}| \leq \delta_{i-1}$, Condition 2, Theorem 1, and the induction hypothesis. Note that Theorem 1 needs $T_{i-1} + (2^\nu - 1)D_{i-1} + e_{i-1} + e_{i-1} \leq 2T_i + (2^\nu - 1)D_{i-1} + e_{i-1} \leq \frac{N}{\beta_{i-1}} - 1 \leq \frac{N}{2} - 1$.

- The remaining proof is identical to Theorem 2, except we need $D_i = D_{i-1}\beta_{i-1}$. We still end with $\bar{M}_i = M'_{i-1} \cdot X^{e_{i-1}\beta_{i-1}}$ and $e_i = e_{i-1}\beta_{i-1} + \psi^*(c_{\text{quo},i}) = \frac{e - \psi^*(c_{\text{rem},i})}{\Delta_i}$.

The correctness of SampleExtract then follows from Condition 4. $\square$

B Comparing TruncRepeat* with Other Packing Methods

Table 9 summarizes the performance of multiple packing methods to compute $\text{truncRepeat}(\cdot, \llbracket -T_i, T_i \rrbracket, \beta_i)$ homomorphically. Note that although TruncRepeat* is more efficient for this special task, it introduces garbage coefficients, which may be undesirable for other packing tasks.

Table 9: The computational cost, evaluation key size, and variance of introduced noise of different packing methods. Time cost is measured in the number of integer multiplications, assuming each FFT in R costs $cN \log N$. Key size is measured in the number of polynomial coefficients.

Packing method	Time cost ^b	Key size ^a	Noise variance (w/o. FFT noise)
TruncRepeat*	$c \lceil \frac{N}{\epsilon+1} \rceil lN \log N$	$2 \lceil \frac{N}{\epsilon+1} \rceil lN$	$(2T_i \lceil \frac{N}{\epsilon+1} \rceil + N) \frac{lB^2\sigma^2}{12} + \frac{Nq^2}{24B^{2l}}$
PubFuncKS	$clN^2 \log N$	$2lN^2$	$\beta_i(2T_i + 1)N \frac{lB^2\sigma^2}{12} + \frac{Nq^2}{24B^{2l}}$
Base-aware PackingKS (scalar mult.)	$(2T_i + 1)lN^2$	$2lN^2$	$(2T_i + 1)N \frac{lB^2\sigma^2}{12} + \frac{Nq^2}{24B^{2l}}$
Base-aware PackingKS (polynomial mult.)	$clN^2 \log N$	$2lN^2$	$(2T_i + 1)N \frac{lB^2\sigma^2}{12} + \frac{Nq^2}{24B^{2l}}$
Automorphism-based packing (power-of-two β_i)	$c(2T_i + \log(N/\beta_i))lN \log N$	$2lN \log N$	$\beta_i \log N (N \frac{lB^2\sigma^2}{12} + \frac{Nq^2}{24B^{2l}})$
Automorphism-based packing (non-power-of-two β_i)	$c(2^\kappa + (\log N - \kappa)\beta_i(2T_i + 1))lN \log N$	$2lN \log N$	$\log N (N \frac{lB^2\sigma^2}{12} + \frac{Nq^2}{24B^{2l}})$

^a The evaluation keys can be shared in different iterations for (a) TruncRepeat* with the same β_i and ϵ_i , (b) base-aware PackingKS with the same β_i , (c) PubFuncKS and automorphism-based packing.

^b κ in the last row is the smaller integer satisfying $2^\kappa > \beta_i(2T_i + 1)$.

We now describe how to obtain the time cost in the last row. With a packing base of one, the automorphism-based packing is given N LWE ciphertexts as inputs and uses a depth- $\log N + 1$ binary tree to combine them into a single RLWE ciphertext. With the leaf node layer as the zeroth layer, the i -th layer includes $N/2^i$ RLWE ciphertexts, where the j -th one contains the input LWE messages with indices in $j + N/2^i \cdot [2^i]$. Each non-leaf node is obtained using a homomorphic automorphism, costing a total of $N - 1$ ones.

To compute truncRepeat , all but the first $\beta_i(2T_i + 1)$ LWE ciphertexts are zero, which means that some nodes in the binary packing tree are encryptions of zero and thus can be pruned. In the i -th layer, there are $\max(0, N/2^i - \beta_i(2T_i + 1))$

zero nodes, accounting to a total of $N - \sum_{i=1}^{\log N - \kappa} (N/2^i - \beta_i(2T_i + 1)) = 2^\kappa + (\log N - \kappa)\beta_i(2T_i + 1)$ nonzero non-leaf nodes. The parameters used in Section 6 satisfy $2^\kappa \geq N/2$, meaning the Meta-PBS needs at least $N/2 + \beta_i(2T_i + 1) > 3N/4$ automorphisms.

C Automatic Parameter Selection

The core workflow of the automatic parameter selection algorithm is described below.

1. c_{meta} , c_{Hom} , and $r_{\text{ext}} = r_K - 2\delta_K$ are provided as inputs.
2. *Estimate K , the number of required iterations.*
First, we find a (not necessarily tight) lower bound of r_K . According to Conditions 1 and 4 of Corollary 4, r_K should satisfy

$$r_K \geq 2\delta_K + r_{\text{ext}} \geq 2\text{erfc}^{-1}(p_{\text{fail}}) \cdot \sqrt{2 \cdot (n/24 + 1/12) + 2(\frac{\alpha_K t}{q})^2 \cdot V_{\text{in}}} + r_{\text{ext}}.$$

Since $2^\nu \cdot \max(r_i, D_i) \leq \alpha_i$, we can replace α_K with $2^\nu \cdot r_K$ to obtain a necessary but insufficient condition for a successful Meta-PBS. The condition can now be transformed into the form of $ar_K^2 + br_K + c \geq 0$ with $a = 1 - 8\text{erfc}^{-1}(p_{\text{fail}})^2 2^{2\nu} (\frac{t}{q})^2 V_{\text{in}}$, $b < 0$ and $c > 0$. If $2^\nu \cdot 2^{-c_{\text{meta}}} < 1$, we know $a > 0$ and thus a solution in the form of $r_K \geq d$ for some $d > 0$ is guaranteed. Then we decide the maximum β_i from r_{i+1} in the reverse order. According to Conditions 2 of Theorem 5, we have

$$\beta_i(2T_i + 1 + (2^\nu - 1)D_i + \epsilon_i) \leq N.$$

Following Conditions 1 and 3, replacing T_i with $\text{erfc}^{-1}(p_{\text{fail}}) \cdot \sqrt{2 \cdot (n/24 + 1/12) + 2(\frac{2^\nu \cdot r_{i+1} t}{q\beta_i})^2 \cdot V_{\text{in}}}$ and D_i with $\frac{2^\nu \cdot r_{i+1}}{\beta_i}$ produces a necessary but insufficient condition of the inequality, which is transformed into a quadratic inequality in β_i . The minimum value of K is estimated by using the maximum β_i at each step.

3. *Enumerate through all valid choices of $\beta \in \bigcup_{j \in [K+2]} \mathbb{Z}^j$ and choose the one with the lowest computational cost.*

The enumeration of β is performed in the reverse order recursively, starting with a value of r_K . Given r_{i+1} , the upper bound of β_i is determined in the same way as in the previous stage, and all β_i s below this bound are enumerated. Then, r_i is set to $\lceil \frac{r_{i+1}}{\beta_i} \rceil$ in the next level of recursion. A choice of β is accepted when $r_i \leq 2^{-\nu} \alpha_0$, i.e., the requirement for the initial redundancy can be satisfied.

Now we have multiple candidates for β , and their validity will be verified using Conditions 1 to 4. This verification is possible because for each fixed choice of β , the minimum value for r_K can be obtained using Condition 4 since α is decided. r_i could also be solved by $r_i = \lceil \frac{r_{i+1}}{\beta_i} \rceil$ for $i \in [K]$. The cost of each valid parameter set is estimated roughly as the number of required polynomial-RLev products.

4. *Decide the gadget parameters for blind rotation, TruncRepeat, and key switching so that the target c_{meta} and c_{Hom} are achieved.*

Note that given l_{br} (or $l_{\text{tr}}/l_{\text{ks}}$), the corresponding best decomposition base can be obtained by solving $\frac{\partial V_{\text{br}}}{\partial l_{\text{br}}} = 0$, so we only need to enumerate all the choices of $(l_{\text{br}}, l_{\text{tr}}, l_{\text{ks}})$. Among all the possible choices of $(l_{\text{br}}, l_{\text{tr}}, l_{\text{ks}})$, we choose the lexicographically smallest one to minimize the computational cost of Meta-PBS.

The full parameter selection algorithm is given in Algorithm 4.

Algorithm 4 Automatic Parameter Selection

Require: Basic PBS parameters, N, n, q, t, ν , and whether the PBS is a WoPPBS

Require: Target homomorphic capacity c_{Hom}^*

Require: S_c , the set of candidates of c_{meta}

Require: S_r , the set of candidates of $r_{\text{ext}} = r_K - 2\delta_K$

Ensure: Parameters for Meta-(WoP)PBS(ManyLUTs) that provides c_{Hom} bits of capacity after bootstrap, with a $c_{\text{meta}} \in S_c$

- 1: $\text{MinCost} \leftarrow +\infty$
 - 2: **for** $c_{\text{meta}}^*, r_{\text{ext}}^* \leftarrow S_c \times S_r$ **do**
 - 3: Run the subroutine described above with $c_{\text{meta}}^*, c_{\text{Hom}}^*, r_{\text{ext}}^*$ as the lower bounds for $c_{\text{meta}}, c_{\text{Hom}}$ and r_{ext} . Denote the obtained parameter set as Params.
 - 4: Estimate the cost of Params as $\text{Cost} = \text{The number of required FFT/iFFTs}$.
 - 5: **if** $\text{Cost} < \text{MinCost}$ **then**
 - 6: $\text{BestParams} \leftarrow \text{Params}, \text{MinCost} \leftarrow \text{Cost}$
 - 7: **end if**
 - 8: **end for**
 - 9: **return** BestParams
-

D Output Noise Distribution

D.1 Distribution of Each Noise Coefficient

Before starting, we note that tfhe-go implements $\text{CMux}(\text{RGSW}(s_i), \text{ACC}, \text{ACC} \cdot X^{-a_i})$ as

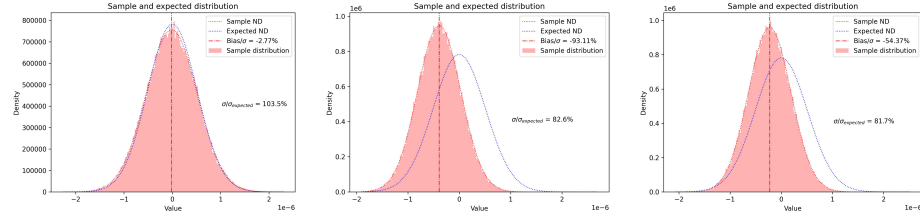
$$\text{ACC} + (\text{ACC} \boxtimes \text{RGSW}(s_i)) \cdot (X^{-a_i} - 1),$$

instead of the standard way of

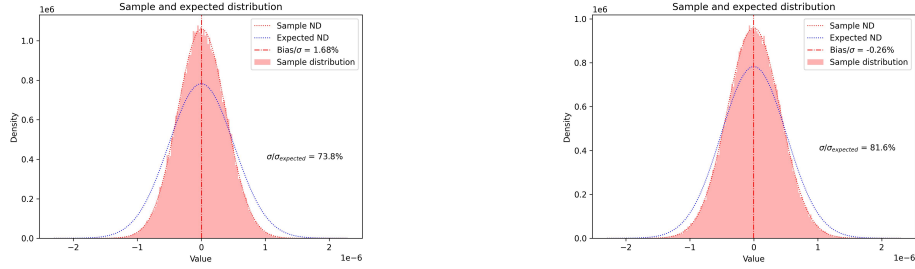
$$(\text{ACC} \cdot (X^{-a_i} - 1)) \boxtimes \text{RGSW}(s_i),$$

as implemented in tfhe-rs. Performing the multiplication by $X^{-a_i} - 1$ after the external product $\boxtimes$ has the side effect of amplifying the external product noise by a factor of two.

However, after fixing this issue, we find that the output noise distribution of Meta-PBS is close to a Gaussian distribution with a nonzero mean and a



(a) Meta-PBS in tfhe-go. (b) Meta-PBS in tfhe-go with new CMux implementation. (c) Three consecutive blind rotations in tfhe-go with new CMux implementation.



(d) Three consecutive blind rotations in tfhe-rs. (e) Meta-PBS in tfhe-go with randomized blind rotation.

Fig. 4: Noise distributions for Meta-PBS or three consecutive blind rotations in tfhe-go and tfhe-rs. The noise is normalized with respect to a ciphertext modulus of one. Each experiment is repeated for 100,000 times.

below-estimation standard deviation (see Figure 4b). To investigate whether this behavior is library-dependent, we perform three consecutive blind rotations using LWE ciphertexts with moduli of $2N$, 5 , and 4 in order in both `tfhe-go` and `tfhe-rs`. One can think of the computation as a 9-bit Meta-PBS without the `TruncRepeat` part. It turns out that output noise bias in `tfhe-rs` is much smaller, but still non-negligible compared to the acceptable bias-to-stddev ratio of 0.3% (see Figure 4c and Figure 4d). Also, the measured standard deviation in `tfhe-rs` is even smaller than that in `tfhe-go`.

An interesting finding is that the noise bias of Meta-PBS decreases as the β_i s increase, and almost disappears when LWE ciphertexts modulo $2N$ are used in all three blind rotations. Thus, we propose an ad-hoc method to remove the bias in the output noise by randomizing ACC between CMux operations. The randomization consists of multiplying the i -th ACC by X^{r_i} , where $r_i \leftarrow \mathbb{Z}_{2N}$ for $i \in \llbracket 0, n-2 \rrbracket$ and $\sum_{i \in \llbracket 0, n-1 \rrbracket} r_i \equiv 0 \pmod{2N}$. The equation above ensures that the blind rotation result still encrypts the same plaintext. Experiments show that the noise is almost unbiased after applying this mitigation (see Figure 4e), and the mitigation overhead is negligible because it only requires a negacyclic rotation of the coefficients of ACC per external product.

We note that the below-estimation output noise variance is not a phenomenon exclusive to Meta-PBS, but is also present in regular PBS. The measured standard deviation of one blind rotation is about 90.1% (or 78.6%) of the estimated one in `tfhe-go` (or `tfhe-rs`). Based on this measurement, the estimated ratio between the measured and expected noise standard deviation after three consecutive blind rotations would be $\sqrt{(1 + \frac{4}{5} + \frac{3}{4})/3} \cdot 90.1\% = 83.1\%$ (or 72.5%) in `tfhe-go` (or `tfhe-rs`).⁹ This agrees with the measured ratios of 81.7% and 73.8%.

It is worth further exploration to find why blind rotations produce biased noises with below-expectation standard deviation, and how this behavior is affected by different implementations.

D.2 Independence of Output Noise Coefficients

Since `truncRepeat` duplicates the coefficients of the noise polynomial in its input RLWE ciphertext, Meta-PBS should have correlated output noise coefficients. However, we argue that the output LWE ciphertexts of Meta-PBSManyLUTs still have independent noise terms.

This is because the duplicated noise coefficients correspond to the same LWE plaintext, and during sample extraction, at most one LWE ciphertext is extracted for the same LWE plaintext. Thus, the duplicated noise terms never appear in more than one output LWE ciphertext. If we additionally assume that blind rotation and `TruncRepeat` produce coefficient-wise independent noise coefficients, which is a common assumption for PBSManyLUTs-based algorithms like circuit

⁹ The noises of the last two blind rotations are multiplied by $\frac{4}{5}$ and $\frac{3}{4}$ respectively here because each coefficient of their input LWE ciphertexts equals zero with a probability of $\frac{1}{5}$ and $\frac{1}{4}$.

bootstrapping, we can treat the output noise terms of Meta-PBSManyLUTs as independent.

As a final remark, although the duplication of the noise coefficients does not affect the independence analysis, it still introduces a slight drawback. Since the noise coefficients corresponding to the same LWE plaintext are highly correlated, the multi-value extraction method [16] cannot be applied to reduce the noise growth in post-bootstrap linear combinations.